

Szegedi Tudományegyetem
Informatikai Intézet

SZAKDOLGOZAT

Kálmán Ádám

2026

Szegedi Tudományegyetem
Informatikai Intézet

NearPick -
Helyalapú mobilalkalmazás fejlesztése

Szakdolgozat

Készítette:
Kálmán Ádám
Programtervező
informatikus szakos
hallgató

Témavezető:
Dr. Bilicki Vilmos
egyetemi docens

Szeged
2026

Feladatkiírás

Az élelmiszer-pazarlás jelentős része abból adódhat, hogy a helyi kereskedőknél a nap végén megmaradó, romlandó, de még fogyasztható termékek nem jutnak el időben a vásárlókhoz.

A feladat egy olyan Flutter [1] alapú, Firebase [2] backendet használó mobilalkalmazás elkészítése, amely összekapcsolja a kedvezményes termékeket kínáló kereskedőket és a közeli és releváns ajánlatokat kereső vásárlókat.

Az alkalmazásban a kereskedők termékeket tudnak feltölteni, szerkeszteni, képpel, mennyiséggel, árral, lejáratí idővel és átvételi időszávvá ellátni. A vásárlók helyalapú és kategória szerinti szűréssal böngészhetik az ajánlatokat, térképes nézetben is megtekinthetik őket, majd foglalást hozhatnak létre. A rendszer támogatssa a többdarabos termék foglalást, az átvételi kód vagy QR-kód alapú ellenőrzést, a lemondási és visszatérítési folyamatokat, valamint az értékelések kezelését. A kereskedők számára készüljön dashboard statisztikákkal, árazási javaslatokkal és exportálható riporttal. A rendszer tartalmazzon szerepkőralapú beléptetést vásárlói, kereskedői és adminisztrátori felületekkel, ahol az adminisztrátor felhasználókat, termékeket és foglalásokat tud felügyelni és moderálni.

Tartalmi összefoglaló

Téma megnevezése:

NearPick – Helyalapú mobilalkalmazás fejlesztése

A megadott feladat megfogalmazása:

A feladat egy olyan mobilalkalmazás elkészítése, amely a helyi kereskedők megmaradt, kedvezményes termékeit összeköti a közelben tartózkodó vásárlókkal. Az alkalmazás célja, hogy segítse az élelmiszer-pazarlás csökkentését, miközben a vásárlók gyorsan és egyszerűen találhatnak közeli, kedvező árú ajánlatokat.

Megoldási mód:

A feladat megoldásához Flutter [1] keretrendszert és Dart [3] programozási nyelvet használok. A backend Firebase [2] alapú szolgáltatásokra épül, többek között Firebase Authentication [4], Cloud Firestore [5], Firebase Storage [6], Cloud Functions [7] és Firebase Messaging [8] használatával.

Alkalmazott eszközök, módszerek:

A fejlesztés Visual Studio Code [9] környezetben, Flutter [1] keretrendszerrel és Dart [3] nyelvvel történt. A backend Firebase [2] szolgáltatásokat használ hitelesítésre, adattárolásra, képfeltöltésre, szerveroldali logikára és értesítések kezelésére.

Elért eredmények:

Az elkészült alkalmazás lehetővé teszi, hogy a vásárlók közeli, kedvezményes ajánlatokat keressenek, kategóriák és helyadatok alapján szűrjenek, majd foglalást hozzanak létre. A kereskedők saját termékeket tölthetnek fel, kezelhetik a foglalásokat, áttekinthetik a főbb statisztikákat, árazási javaslatokat kaphatnak, valamint exportálható riportokat készíthetnek.

Kulcsszavak:

Élelmiszer-pazarlás, helyi kereskedők, mobilalkalmazás, foglalás, Flutter [1], Dart [3], Firebase [2], QR-kód, marketplace

Tartalomjegyzék

Feladatkiírás	1
Tartalmi összefoglaló	2
Bevezetés.....	5
1. Probléma és piaci háttér	6
1.1. Vásárlói és kereskedői igények.....	6
1.2. Hasonló alkalmazások áttekintése	7
1.3. A NearPick célja és pozicionálása	8
2. A NearPick koncepciója és szereplői.....	10
2.1. A többoldalú piactér működése.....	10
2.2. Vásárlói szerepkör	10
2.3. Kereskedői szerepkör.....	11
2.4. Adminisztrátori szerepkör.....	11
2.5. A szereplők közötti kapcsolat	12
3. Felhasználói folyamatok és használati esetek.....	13
3.1. Vásárlói folyamatok.....	13
3.2. Kereskedői folyamatok	16
3.3. Adminisztrátori folyamatok.....	19
3.4. A fő folyamatok kapcsolódása	21
4. Technológiai háttér és rendszerarchitektúra.....	22
4.1. Technológiai választások	22
4.2. A rendszer magas szintű felépítése	23
4.3. A Kliensalkalmazás felépítése	24
4.4. Hitelesítés és szerepköralapú működés.....	25
4.5. Cloud Firestore és adattárolás	25
4.6. Firebase Storage és termékképek.....	25

4.7.	Cloud Functions és szerveroldali üzleti logika	26
4.8.	Firebase Cloud Messaging és értesítések.....	26
5.	Adatmodell és jogosultságkezelés	27
5.1.	Az adatmodell felépítésének alapelvei	27
5.2.	A rendszer fő entitásai.....	28
5.2.1.	Felhasználók	28
5.2.2.	Termékek.....	28
5.2.3.	Foglalások.....	28
5.2.4.	Értékelések.....	29
5.2.5.	Kiegészítő entitások.....	29
5.3.	Jogosultságkezelési modell.....	29
5.4.	Firestore és Storage szabályok.....	29
6.	Megvalósítás fontosabb részletei	31
6.1.	Az ajánlórendszer működése	31
6.2.	A foglalási folyamat és készletkezelés.....	33
6.3.	QR-kódos és átvételi kódos teljesítés	36
6.4.	Adminisztrátori műveletek megvalósítása.....	38
7.	Tesztelés és minőségbiztosítás	41
7.1.	Kliensoldali tesztelés	41
7.2.	Backend és jogosultsági tesztek.....	41
7.3.	Automatizált minőségkapuk	42
8.	Mesterséges intelligencia használata a fejlesztés során	43
9.	Továbbfejlesztési lehetőségek.....	45
	Irodalomjegyzék.....	47
	Nyilatkozat.....	48
	Elektronikus mellékletek.....	49

Bevezetés

Mindig látni a boltokban leárazásokat olyan termékekre, amelyek közel járnak a lejárat idejükhöz. Ez kedvező ajánlat lehet, ha éppen az adott élelmiszerre van szükségünk, azonban erről többnyire csak akkor tájékozódhatunk, ha éppen ott vagyunk az üzletben. Innen jött az ötlet, hogy készülhetne egy alkalmazás, ami ezt a folyamatot hatékonyabbá teszi, és megkönnyíti mind a kereskedők, mind a vásárlók dolgát. Emellett az alkalmazás használatával csökkenteni lehetne az élelmiszer-pazarlást, mert ami a lejárat előtt nem talál gazdára, azt később jó eséllyel kidobják. A vásárlók ezzel spórolhatnának is, ugyanis ha olyan termék áll rendelkezésre, amelyre szükségük van, akkor kedvezőbb áron tudják azt beszerezni, mintha teljes áron vásárolnák meg. A kereskedők fel tudják tölteni a közeljövőben lejárando termékeket, a vásárlók pedig anélkül, hogy körbejárnák az összes kereskedést akciós termékek után kutatva, csak megnézik az alkalmazásban, hogy van-e a számukra megfelelő ajánlat. Ehhez segítséget nyújt a helyalapú ajánlás, a kategóriaszűrés és az ajánlórendszer, ami belső adatok alapján rendezi a felhasználó számára várhatóan releváns termékeket.

Személy szerint korábban nem ismertem ilyen jellegű megvalósítást, a saját környezetemben pedig nem találkoztam hasonló szolgáltatással. Ennek ellenére természetesen végeztem piackutatást, amely során találtam ehhez hasonló alkalmazásokat. Ezek közül több csomagalapú megoldást kínál, illetve egyes rendszerek inkább nagyobb üzletláncokra fókuszálnak. Nem mindegyik érhető el minden országban, és a személyre szabhatóságuk is korlátozott lehet. Ezek figyelembe vételével próbáltam kialakítani a NearPick alkalmazást. Törekedtem az egyszerű használhatóságra, hogy fiatalabb és az idősebb korosztály számára se legyen bonyolult a termékfoglalás.

A vásárló bejelentkezés után rögtön láthatja a számára releváns termékeket. Térképes nézetre is válthat, így áttekintheti, hogy hozzá képest milyen messze is vannak az egyes üzletek. Beállíthatja az érdekelt kategóriákat, és ezek alapján később push értesítést kaphat releváns új ajánlatokról, hogy véletlen se maradjon le semmiről. A lefoglalni kívánt termékre kattintva megtekintheti a részleteket, majd a foglalás gombbal lefoglalhatja azt. Ezután megjelenik számára az átvételi kód és a QR-kód, amit csak bemutat a kereskedőnek, és már kész is. Ezután értékelheti az üzletet vagy a kereskedőt, ezzel segítve a kereskedők munkáját és a többi vásárló döntését. Ha mégsem szeretné megvásárolni a terméket, akkor az alkalmazáson belül lemondhatja a foglalást.

1. Probléma és piaci háttér

Az élelmiszer-pazarlás mind a háztartásokban, mind a kiskereskedelmi szektorban jelen lévő probléma. A boltokban, pékségekben, vendéglátóhelyeken és kisebb helyi kereskedőknél gyakran maradnak meg olyan termékek, amelyek még fogyaszthatók, de rövid időn belül elveszítik értéküket. Ezek lehetnek közeli lejáratú élelmiszerek, nap végén megmaradt pékáruk, szezonális termékek vagy olyan áruk, amelyekből a vártnál nagyobb készlet maradt. Ha ezek időben nem találunk vásárlóra, akkor a kereskedő számára veszteséget jelentenek, később pedig sok esetben hulladékká válnak.

A probléma nem kizárólag környezetvédelmi szempontból jelentős. A kereskedők oldaláról gazdasági kérdés is, hiszen a megmaradó készlet csökkenti a bevételt és növeli a veszteséget. A vásárlók oldaláról ugyanakkor lehetőséget jelent, mert a közelgő lejáratú vagy rövid ideig elérhető termékek kedvezményes áron is vonzóak lehetnek. A nehézséget az okozza, hogy ezek az ajánlatok sokszor csak helyben, az adott üzletben derülnek ki. Egy vásárló általában nem járja végig a környék összes kereskedését azért, hogy megtudja, hol milyen leértékelt termék érhető el.

Ebből következik a probléma megoldása. Valahogy informálni kell a vásárlókat, és ez még időben történjen meg. Ha a kereskedő gyorsan fel tudja tölteni a megmaradt terméket, a vásárló pedig hely, kategória és relevancia alapján azonnal megtalálja azt, az mindkét félnek a hasznára válik. A kereskedő csökkentheti a veszteségét, a vásárló kedvezőbb áron juthat hozzá számára hasznos termékhez.

1.1.Vásárlói és kereskedői igények

Egy ilyen rendszer tervezésénél fontos külön vizsgálni a vásárlói és a kereskedői oldalt. A vásárló elsődleges célja, hogy gyorsan, egyszerűen és átlátható módon találjon számára releváns ajánlatokat. Nem elegendő csupán egy hosszú terméklista megjelenítése, mert a túl sok irreleváns ajánlat rontja a használhatóságot. Lényeges, hogy a foglalási folyamat egyszerű legyen, és az átvételkor ne legyen szükség bonyolult egyeztetésre.

A kereskedő oldalán ezzel szemben a gyors termékfeltöltés, az egyszerű foglaláskezelés és az áttekinthető adminisztráció a legfontosabb. A rendszer akkor tud

valódi segítséget nyújtani, ha a kereskedő néhány lépésben fel tud tölteni egy ajánlatot, majd később könnyen ellenőrizni tudja, hogy ki mit foglalt le.

A piactér jellegű működés miatt szükség van egy harmadik szereplőre is: az adminisztrátorra. Az adminisztrátori szerepkör feladata, hogy a rendszer működését felügyelje, a problémás felhasználókat vagy termékeket kezelje, és szükség esetén beavatkozzon.

1.2. Hasonló alkalmazások áttekintése

A NearPick megtervezése előtt érdemes volt utánanézni a már létező, hasonló célú megoldásoknak. A piacon több olyan alkalmazás is található, amely az élelmiszer-pazarlás csökkentésével és a kedvezményes ajánlatok közvetítésével foglalkozik. Ilyen például a Too Good To Go [10], a Munch [11] vagy a FoodHero [12]. Ezek az alkalmazások különböző módon közelítik meg ugyanazt a problémát: vannak, amelyek előre összeállított csomagokat kínálnak, mások termékszintű kedvezményekkel dolgoznak, illetve vannak olyan megoldások, amelyek elsősorban nagyobb kereskedelmi láncokat céloznak meg.

A Too Good To Go [10] egyik erőssége a nemzetközi ismertség és az egyszerű felhasználói élmény. A szolgáltatás jellemzően csomagalapú működésre épít, ahol a vásárló nem feltétlenül konkrét terméket választ, hanem egy előre összeállított vagy meglepetésszerű csomagot foglal le. Ez egyszerűvé teszi a kereskedői oldalt, azonban a vásárlói személyre szabhatóságot korlátozhatja.

A Munch [11] a magyar piacon is ismert, amely szintén az élelmiszermentés gondolatára épül. Erőssége, hogy helyi felhasználók számára is elérhető, és közvetlen kapcsolatot teremt a kereskedők és a vásárlók között. Ugyanakkor a piackutatás alapján itt is jellemzőbb az előre összeállított csomagok használata, ami nem minden esetben ad teljes kontrollt a vásárlónak a konkrét termék kiválasztásában.

A FoodHero [12] alkalmazás elsősorban statikus, termékszintű kedvezményeket kínáló platformként működik, amelyben a felhasználók a kereskedők által feltöltött ajánlatok közül válogathatnak. Ezzel szemben a NearPick egy személyre szabott ajánlórendszert alkalmaz, amely a felhasználói preferenciák, a földrajzi távolság, valamint a termékek jellemzői alapján rangsorolja a kínálatot. Ennek eredményeként a NearPick nem csupán megjeleníti az elérhető termékeket, hanem támogatja a felhasználói döntéshozatalt is.

A meglévő megoldások alapján látható, hogy a probléma valós, és már többféle alkalmazás próbál választ adni rá. Marad viszont tér olyan megközelítésre, amely a kisebb helyi kereskedőket, a termékszintű ajánlatokat, a helyalapú böngészést és a személyre szabottabb vásárlói élményt helyezi előtérbe.

Alkalmazás	Fő működés	Erősség	Korlát
Too Good To Go	csomagalapú ételmentés (meglepetés csomagok)	széles körben elterjedt, egyszerű használat, nagy felhasználói bázis	nincs konkrét termékválasztás, alacsony kontroll a tartalom felett
Munch	csomagalapú ételmentés	lokálisan ismert, hazai piacon releváns	korlátozott személyre szabás, hasonló modell mint a Too Good To Go
FoodHero	termékszintű kedvezmények boltokon belül	pontos termékválasztás, valós készlethez közeli működés	főként nagy kereskedelmi láncokra épít, nincs személyre szabott ajánlás
NearPick	termékszintű, helyalapú foglalás ajánlórendszerrel	személyre szabott ajánlás, kisebb kereskedők bevonása, távolság-alapú rangsorolás, foglalási rendszer	prototípus jelleg, korlátozott elérhetőség, kisebb adatbázis

1.2.1 ábra: Piaci áttekintés

1.3. A NearPick célja és pozicionálása

A NearPick célja egy olyan mobilalkalmazás létrehozása, amely a helyi kereskedők időérzékeny, kedvezményes termékeit összekapcsolja a közelben tartózkodó vásárlókkal. A rendszer nem kizárólag az ajánlatok megjelenítésére koncentrál, hanem a teljes folyamatot támogatja a termék feltöltésétől a foglaláson át az átvételig és az értékelésig.

A NearPick egyik fontos sajátossága a termékszintű működés. A vásárló nem csak egy ismeretlen tartalmú csomagot lát, hanem konkrét termékek között böngészhet. Megtekintheti a termék kategóriáját, árát, mennyiségét, lejáratát, átvételi adatait és a kereskedőhöz kapcsolódó információkat. Ez indokoltabb döntést tesz lehetővé, különösen akkor, ha a vásárlónak konkrét igényei vannak.

A rendszer másik fontos eleme a helyalapú és preferenciaalapú ajánlás. A felhasználó számára nem minden termék egyformán releváns: számít, hogy milyen messze van az üzlet, milyen kategóriák érdeklik, mennyire friss az ajánlat, illetve mennyi idő van hátra a termék lejáratáig. A NearPick ezek alapján tudja rendezni és szűrni az ajánlatokat, így a vásárló gyorsabban találhat számára megfelelő terméket.

A kereskedői oldal szintén lényeges része a koncepciónak. Az alkalmazás lehetőséget ad termékek feltöltésére, módosítására, foglalások kezelésére, QR-kódos vagy átvételi

kódos teljesítésre, valamint kereskedői statisztikák megtekintésére. Ezáltal a NearPick nem csak vásárlói alkalmazásként, hanem kereskedői eszközként is működik.

A harmadik fő terület az adminisztráció és moderáció. Egy több szereplős piactér esetén fontos, hogy a rendszerben megjelenő felhasználók, termékek és foglalások felügyelhetők legyenek. A NearPick adminisztrátori felülete ezt a célt szolgálja: áttekintést ad a rendszer állapotáról, lehetőséget biztosít felhasználói fiókok kezelésére, termékek moderálására és kereskedőknek küldhető adminisztrátori üzenetek létrehozására.

Összességében a NearPick egy olyan, Flutter [1] és Firebase [2] alapokra épülő piactér alkalmazás, amely a vásárlói ajánlatkeresést, a kereskedői termékkezelést és az adminisztrátori felügyeletet egy rendszerben valósítja meg. A cél nem egy teljes országos kereskedelmi platform létrehozása volt, hanem egy működő, bővíthető prototípus elkészítése, amely bemutatja, hogyan támogatható digitális eszközökkel a helyi, időérzékeny élelmiszer-ajánlatok kezelése.

2. A NearPick koncepciója és szereplői

Az alkalmazás célja nem pusztán az, hogy termékeket listázzon, hanem hogy végigvezesse a felhasználókat a teljes folyamaton: a kereskedői termékfeltöltéstől kezdve a vásárlói foglaláson át egészen az átvételig és az értékelésig.

A rendszer működésének középpontjában három fő szerepkör áll: a vásárló, a kereskedő és az adminisztrátor. A vásárló ajánlatokat keres és foglal, a kereskedő termékeket tölt fel és foglalásokat kezel, az adminisztrátor pedig a piactér működését felügyeli.

2.1. A többoldalú piactér működése

A NearPick többoldalú piactérként működik, mert a rendszerben több, egymástól eltérő célú felhasználói csoport vesz részt. A vásárlók a kedvező árú, közeli ajánlatokat keresik, a kereskedők a megmaradt termékeiket szeretnék értékesíteni, az adminisztrátor pedig a rendszer működését és megbízhatóságát felügyeli.

A több szerepkör miatt a rendszer tervezésénél fontos szempont volt, hogy minden felhasználó csak a számára releváns funkciókat lássa. A vásárlónak nincs szüksége termékfeltöltő felületre, a kereskedőnek nincs szüksége vásárlói feedre, az adminisztrátornak pedig olyan áttekintő és moderációs funkciókra van szüksége, amelyek a másik két szerepkör számára nem elérhetők. Ezért az alkalmazás szerepköralapú felépítést használ.

2.2. Vásárlói szerepkör

Bejelentkezés után a vásárló egy terméklistát lát, amelyben az aktív ajánlatok jelennek meg. A listában szereplő termékek kategória, helyadat, lejáratidő és egyéb belső szempontok alapján rendezhetők és szűrhetők.

A vásárló számára fontos funkció a kategóriák megadása, ha csak egy konkrét csoporteggyüttes érdekli. A helyalapú működés szintén lényeges, mivel a vásárló általában olyan ajánlatokat keres, amelyeket könnyen és rövid időn belül át tud venni. A térképes nézet ezt támogatja azzal, hogy vizuálisan is megmutatja az ajánlatok elhelyezkedését.

A kiválasztott termék részletes oldalán megtekintheti az ajánlat adatait, majd foglalást hozhat létre. A foglalás után az alkalmazás átvételi kódot és QR-kódot biztosít, amelyet a vásárló az üzletben bemutathat. Szükség esetén lemondhatja a foglalását, teljesített átvétel után pedig értékelést adhat a kereskedőnek. Ez az értékelési folyamat segíti a többi vásárlót, és visszajelzést ad a kereskedőnek is.

2.3. Kereskedői szerepkör

A kereskedő saját profillal rendelkezik, ahol megadhatja a vállalkozásához kapcsolódó alapadatokat, például a cégnevet és a céghelyet. Ezek az adatok a termékfeltöltés során is felhasználhatók, így nem szükséges minden új terméknel újra megadni ugyanazokat az információkat.

Új termék létrehozásakor megadhatja a termék nevét, kategóriáját, árát, eredeti árát, mennyiségét, lejárat idejét, átvételi időszávját és opcionálisan képet is tölthet fel. A termék az aktív ajánlatok között jelenik meg, ahol a vásárlók megtalálhatják és lefoglalhatják. A termék az első foglalás előtt még módosítható, később viszont már nem.

A termék átvétele során QR-kód vagy manuális átvételi kód segítségével ellenőrizheti, hogy a vásárló valóban jogosult-e az adott foglalás átvételére. A rendszer támogatja a foglalási állapotok kezelését, a lemondások és visszatérítési állapotok követését, valamint a teljesített foglalások utáni értékelések megjelenítését.

2.4. Adminisztrátori szerepkör

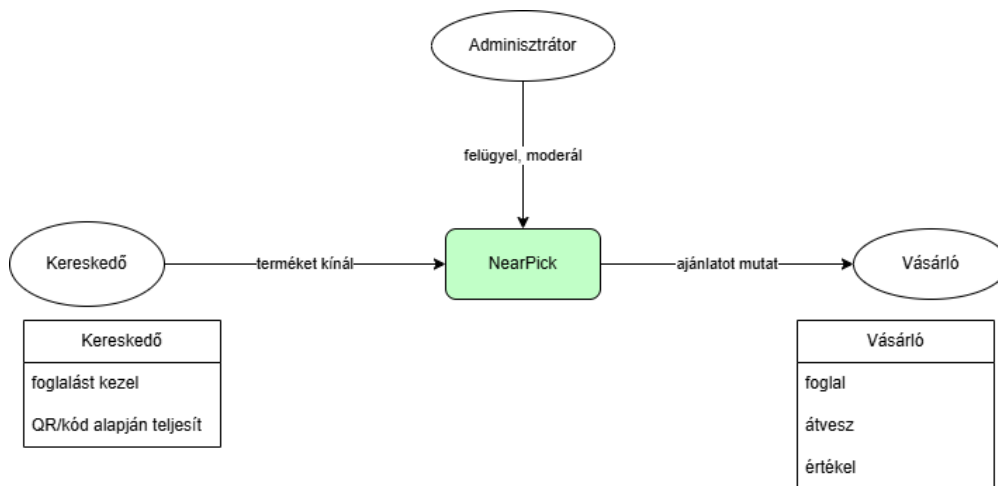
Az adminisztrátor áttekintő felületen láthatja a rendszer főbb adatait, például a felhasználók, kereskedők, vásárlók, termékek és foglalások számát. Emellett kereshet a felhasználók, termékek és foglalások között, megtekintheti azok részleteit, és módosíthatja bizonyos státuszaikat. A felhasználói fiókok esetében például lehetőség van aktív, felfüggesztett vagy blokkolt állapot kezelésére.

A termékmódoráció szintén az adminisztrátori szerepkör része. Elrejthet, visszaállíthat vagy archiválhat termékeket, ha azok nem megfelelőek vagy valamilyen okból problémásak. Emellett az admin közvetlen üzenetet is küldhet a kereskedőknek, amelyet a kereskedő a saját felületén megtekinthet.

2.5. A szereplők közötti kapcsolat

A kereskedő feltölt egy terméket, amely megjelenik a vásárlói oldalon. A vásárló megtalálja az ajánlatot, lefoglalja, majd az átvétel során bemutatja az átvételi kódot vagy QR-kódot. A kereskedő teljesíti a foglalást, a vásárló pedig később értékelést adhat. Ha a folyamat során probléma merül fel, az adminisztrátor felügyelheti és kezelheti az érintett felhasználókat, termékeket vagy foglalásokat.

A következő fejezetekben a dolgozat nem csupán az egyes képernyőket mutatja be, hanem a teljes működési folyamatot, az adatmodellt, a jogosultságkezelést és a megvalósítás fontosabb részleteit is.



2.5.1 ábra: Felhasználók közötti kapcsolat

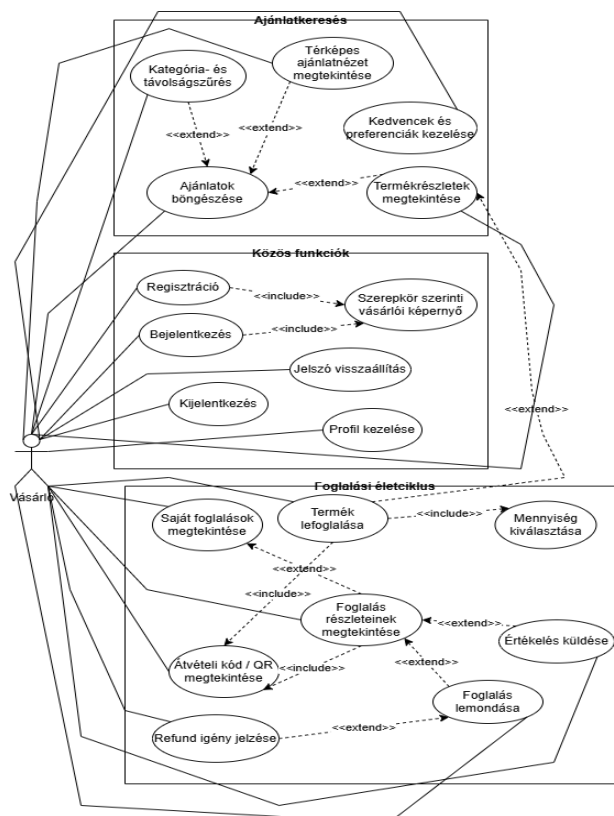
3. Felhasználói folyamatok és használati esetek

Ebben a fejezetben a különböző felhasználók funkciói kerülnek bemutatásra néhány képernyőképpel és eset diagrammal kiegészítve.

Szerepkör	Fő cél	Fontosabb funkciók
Vásárló	Közele, kedvezményes ajánlatok keresése és foglalása	böngészés, szűrés, térkép, foglalás, QR-kód, lemondás, értékelés
Kereskedő	Megmaradt termékek gyors közzététele és foglalások kezelése	termékfeltöltés, szerkesztés, foglaláskezelés, QR ellenőrzés, dashboard, CSV export
Adminisztrátor	A piactér működésének felügyelete	felhasználókezelés, termékmoderáció, foglalás-áttekintés, admin üzenetek

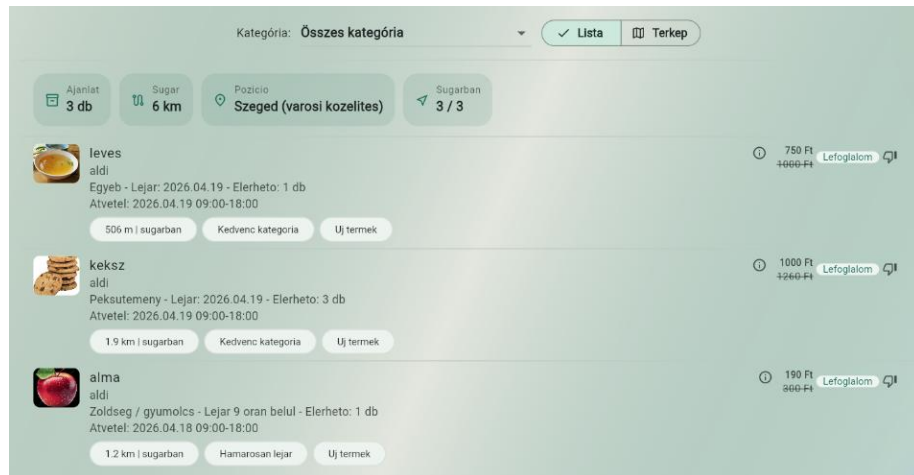
3.1 ábra: Funkcionális áttekintés

3.1. Vásárlói folyamatok



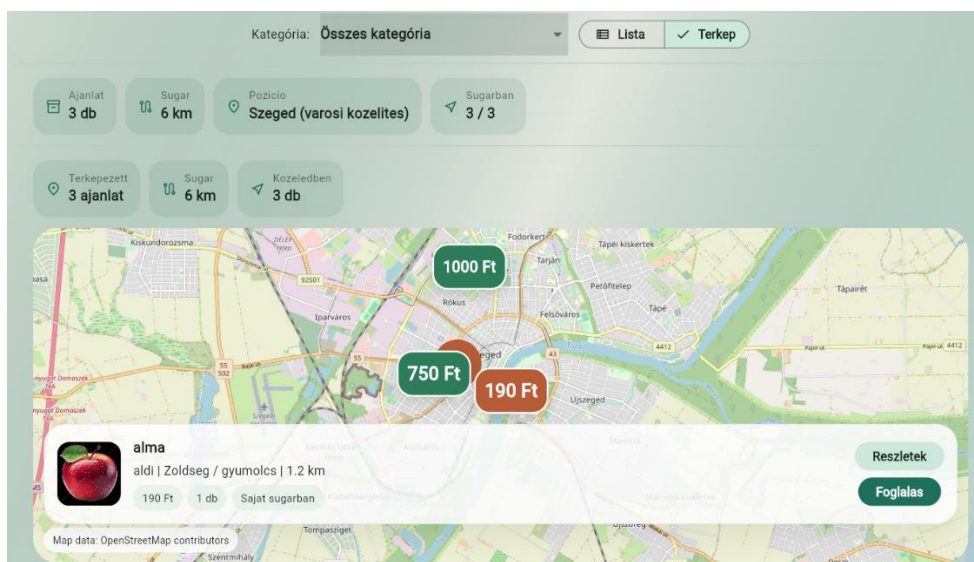
3.1.1 ábra: Vásárlói használati eset diagram

A vásárlói folyamat a bejelentkezéssel kezdődik. A sikeres azonosítás után a fogyasztói kezdőképernyőre kerül, ahol az aktív ajánlatok listáját látja. Az ajánlatok megjelenítésénél fontos szerepet kapnak a felhasználó preferenciái, a termék kategóriája, lejárat ideje és helyzete. A vásárló így nem egy rendezetlen terméklistát kap, hanem olyan ajánlatokat láthat, amelyek nagyobb eséllyel relevánsak számára.



3.1.2 ábra: Vásárlói kezdőoldal

A vásárló kategória szerint szűrheti az ajánlatokat, illetve térképes nézetre is válthat. A térképes megjelenítés hasznos lehet, ha vizuálisan látni akarja a termékek helyzetét, így jobban fel tudja mérni a távolságokat.



3.1.3 ábra: Vásárlói térképes nézet

A termék részletes oldalán a vásárló megtekintheti az ajánlat főbb adatait: a termék nevét, kategóriáját, árát, mennyiségét, lejárat idejét, átvételi időszávját és a kereskedő

adatait. Ha az ajánlat megfelelő, a felhasználó foglalást hozhat létre. A foglalás során a rendszer figyelembe veszi az elérhető mennyiséget, így nem enged olyan foglalást létrehozni, amely meghaladná a rendelkezésre álló készletet.

A sikeres foglalás után a vásárló megkapja az átvételhez szükséges adatokat. Ezek közé tartozik az átvételi kód és a QR-kód, amelyet az üzletben bemutathat a kereskedőnek. Az átvétel után a foglalás teljesített állapotba kerülhet, majd a vásárló értékelést adhat.

Megvalósított funkció a foglalás lemondása is. Ha a felhasználó mégsem tudja vagy nem szeretné átvenni a terméket, az alkalmazáson belül lemondhatja a foglalást.

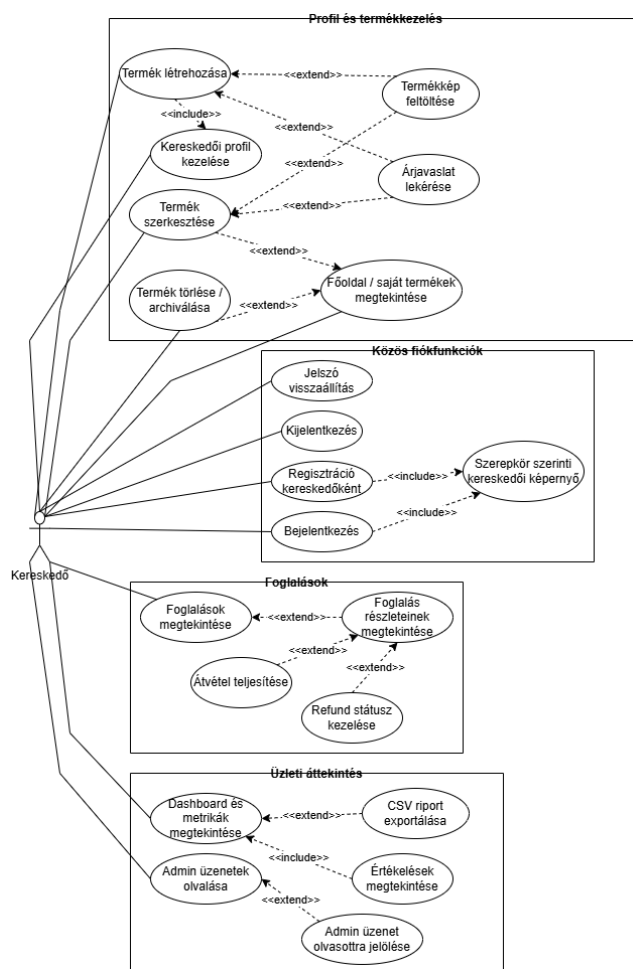


3.1.4 ábra: Foglalási kód és QR kód

Van külön Foglalások menüpont, ahol az eddigi összes foglalását láthatja, egyre rákattintva tekintheti meg annak részleteit. Kedvencek jelölhet termékeket, majd később ezek is megtekinthetők a Kedvencek menüpont alatt.

A profil oldalon áttekintheti a saját adatait. Megjelenik az email címe, felhasználóneve, amit módosíthat is. Itt tudja beállítani a helyadatot, ahol két opció közül választhat. Vagy megadja a pontos helyét, vagy ha ezt nem szeretné, akkor tud választani előre megadott városok közül is. Ehhez kapcsolódik a keresési sugár megadása, hogy mekkora területen belül preferálja a találatokat. Itt kapott helyet még az érdeklődési kategóriák megadása is.

3.2. Kereskedői folyamatok

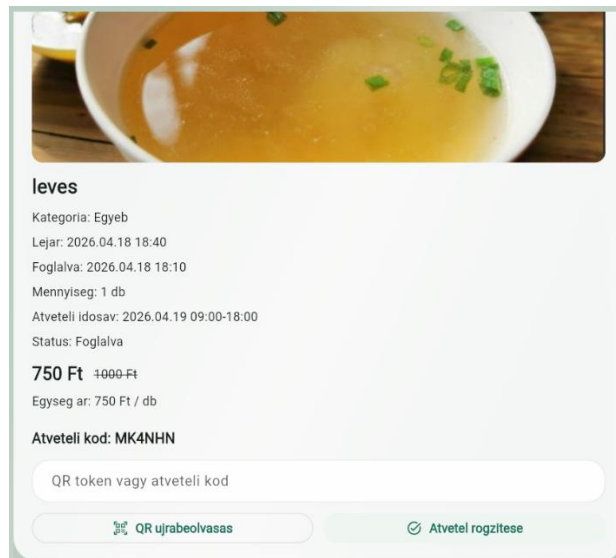


3.2.1 ábra: Kereskedői használati eset diagram

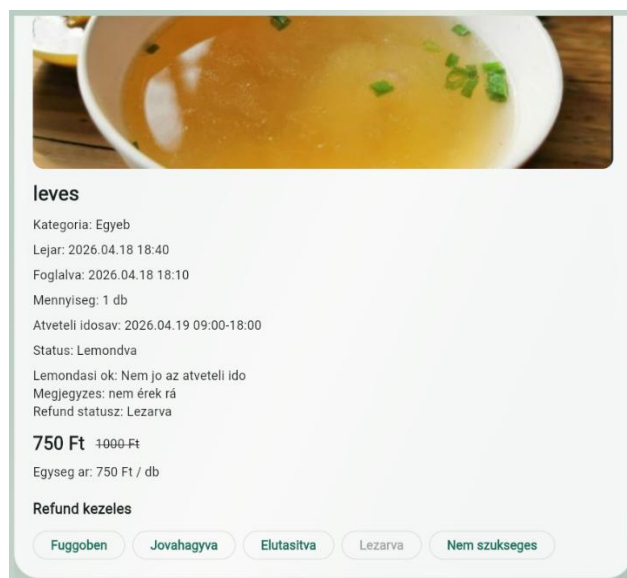
A kereskedő bejelentkezés után a saját felületére kerül, ahol az eddig általa feltöltött termékeket látja, és itt tud új terméket is létrehozni.

Új termék létrehozásakor a kereskedő megadja a termék alapadatait. Ilyen adat a név, kategória, ár, eredeti ár, mennyiség, lejárat idő, átvételi időszak és opcionálisan a termékkép. A validáció célja, hogy hiányos vagy hibás adatokkal ne jöhessen létre ajánlat.

A kereskedő külön fülön követheti a foglalásokat. Az átvétel során a vásárló által bemutatott QR-kód vagy átvételi kód alapján teljesítheti a foglalást. Ez csökkenti a téves átvételek esélyét, és egyszerűbbé teszi a helyszíni folyamatot. A rendszer támogatja a lemondások és visszatérítési állapotok kezelését is, így a foglalási életciklus több állapota is követhető.



3.2.2 ábra: Kereskedői foglalás



3.2.3 ábra: Visszatérítés kezelése

A termék lemondásához egy kis magyarázat. Ha a vásárló kér visszatérítést, akkor a refund státusz automatikusan függőben lesz. Ha jóváhagyottra állítja, akkor elfogadta a visszatérítési igényt, de még nem tekinti lezártnak. Az elutasításkor nem hagyja jóvá. Ha lezárja, akkor lezárja a teljes folyamatot. A nem szükséges státusz azt jelenti, hogy ehhez a termékhez nem szükséges visszatérítés, nincs rá igény. Ezek főleg online fizetéskor lennének fontosak, ezek most csak nyilvántartási állapotok.

A kereskedői oldal része a dashboard is, amely áttekintést ad a kereskedő számára fontos adatokról. Ide tartoznak a foglalásokkal, értékelésekkel, termékekkel és árazási

javaslatokkal kapcsolatos információk. A CSV export lehetőséget ad arra, hogy a kereskedő a saját adatait később külső eszközben is feldolgozhassa.



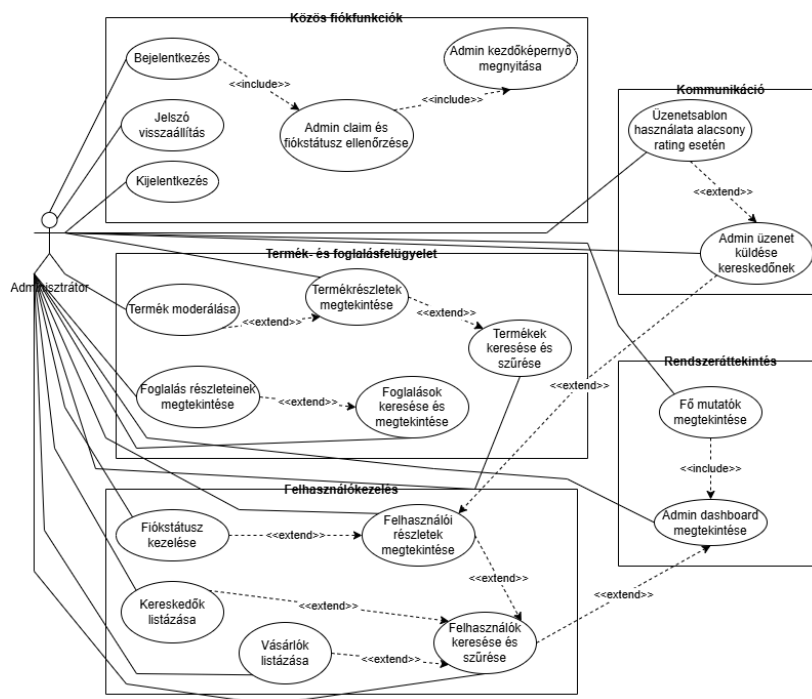
3.2.4 ábra: Kereskedői dashboard részlet

A kereskedő a profil fölön tudja megadni a céghelyet és módosítani a cégnevet, ezek az adatok termékfeltöltés során is felhasználhatók, így a termékek létrehozása gyorsabbá válik, nem kell mindegyiknek megadni külön-külön.

The screenshot shows a 'Kereskedo profil' (Merchant profile) form. It contains three input fields: 'Felhasználonev' (Username) with the value 'merchant', 'Email' with the value 'merchant@gmail.com', and 'Ceg neve' (Company name) with the value 'aldi'. Each field has an edit icon to its right. Below these fields is a 'Hely beallitasa' (Location settings) section. It features a button 'Aktualis hely meghatarozasa' (Set current location) and two input fields for 'Latitude' (46.248199) and 'Longitude' (20.155165). A note below these fields states 'Ez a hely automatikusan hozzáadodik az új termékekhez.' (This location will be automatically added to the new products). At the bottom of the section is a 'Mentes' (Save) button.

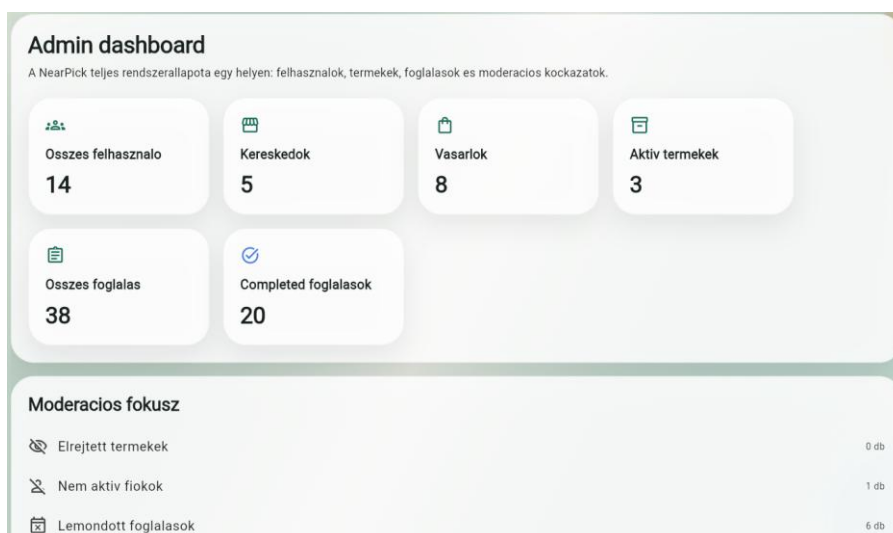
3.2.5 ábra: Kereskedői profil

3.3. Adminisztrátori folyamatok



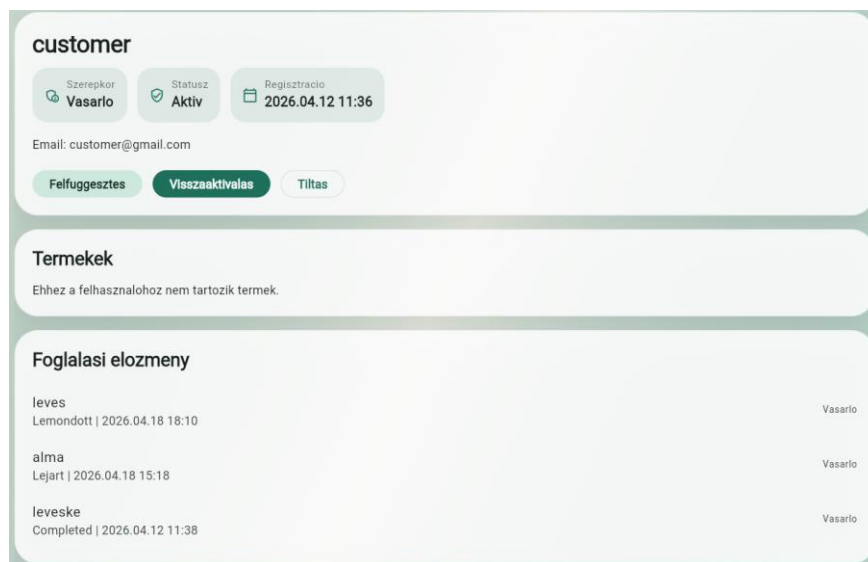
3.3.1 ábra: Adminisztrátori használati eset diagram

Az adminisztrátor bejelentkezés után admin felületre kerül, ahol dashboard jellegű áttekintést kap. Láthatja a felhasználók, kereskedők, vásárlók, termékek és foglalások főbb adatait. Ez az áttekintés segíti abban, hogy gyorsan képet kapjon a rendszer állapotáról.



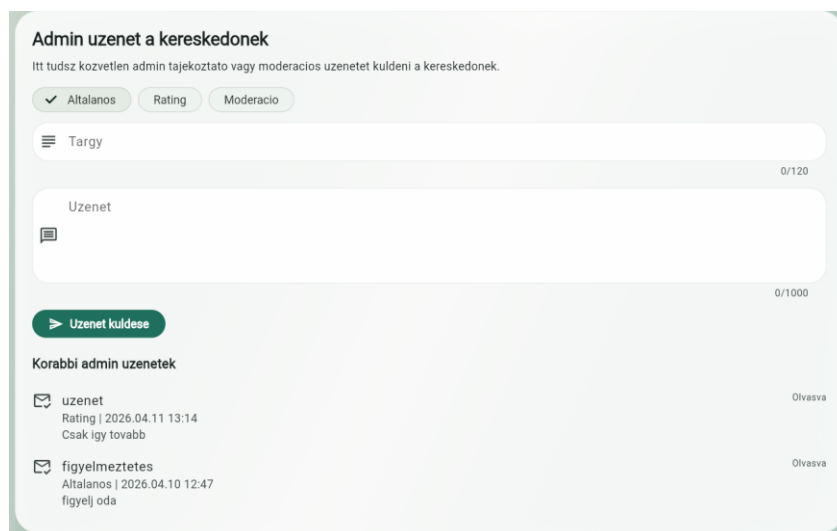
3.3.2 ábra: Admin dashboard

A felhasználókezelés során az adminisztrátor megtekintheti a felhasználók adatait, és módosíthatja a fiókok státuszát. Erre akkor lehet szükség, ha egy fiók problémás viselkedést mutat, vagy valamilyen okból korlátozni kell a hozzáférését. Az adminisztrátor kezelheti a termékeket is: elrejtheti, visszaállíthatja vagy archiválhatja azokat.



3.3.3 ábra: Vásárlói felhasználó részletei admin oldalon

Az adminisztrátori felület foglалás-áttekintést is tartalmaz. Ez lehetőséget ad arra, hogy az adminisztrátor ellenőrizze a rendszerben létrejött foglалásokat, azok állapotát és részleteit. Emellett az admin közvetlen üzenetet küldhet a kereskedőknek, amely a kereskedői felületen jelenik meg. Ez hasznos lehet figyelmeztetés, tájékoztatás vagy moderációs visszajelzés esetén.

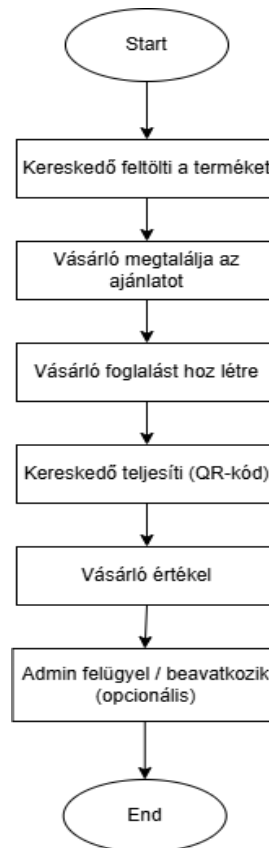


3.3.4 ábra: Admin üzenet küldése

3.4. A fő folyamatok kapcsolódása

A három szerepkör folyamatai egymásra épülnek. A kereskedő létrehoz egy terméket, amely megjelenik a vásárlói oldalon. A vásárló megtalálja és lefoglalja az ajánlatot. A foglalás megjelenik a kereskedői oldalon, ahol a kereskedő az átvételi kód vagy QR-kód alapján teljesítheti azt. A teljesítés után a vásárló értékelést adhat, amely visszajelzéseként szolgál a kereskedőnek és információként a többi vásárlónak. Az adminisztrátor a folyamat bármely pontján áttekintheti a rendszer működését, és szükség esetén beavatkozhat.

A vásárlói, kereskedői és adminisztrátori funkciók együtt biztosítják, hogy a kedvezményes termékek feltöltése, megtalálása, lefoglalása és átvétele egy egységes rendszerben történjen.



3.4.1 ábra: fő üzleti folyamatok

4. Technológiai háttér és rendszerarchitektúra

A NearPick megvalósítása során egy olyan technológiai környezet került kialakításra, amely lehetővé teszi a gyors fejlesztést, az egyszerű üzemeltetést, valamint a valós idejű adatkezelést. Az alkalmazás több szereplős, ahol vásárlók és kereskedők interakcióba lépnek egymással, ezért fontos volt az adatok folyamatos szinkronizációja és a skálázható backend.

4.1. Technológiai választások

A kliensoldali alkalmazás fejlesztéséhez a Flutter [1] keretrendszer került kiválasztásra. A döntés többek között azért erre esett, mert a Flutter [1] lehetővé teszi egyetlen kódbázisból több platformra történő fejlesztést, ami csökkenti a fejlesztési időt és az implementáció komplexitását. Ez előnyös volt az alkalmazása esetében, mivel az alkalmazás több, funkcionálisan eltérő felületet tartalmaz (például vásárlói feed, térképes nézet, termék részletek, foglalási felület, valamint kereskedői és adminisztrátori nézetek), amelyek egységes megjelenítése és kezelése fontos szempont volt.

A Flutter [1] további előnye a gyors fejlesztési ciklus, amelyet a beépített hot reload mechanizmus biztosít. Így a felhasználói felület módosítása azonnal ellenőrizhető, ami megkönnyíti a fejlesztést.

A Flutterhez [1] tartozó Dart [3] programozási nyelvet használtam, ami egyébként is hatékonyan támogatja az aszinkron műveletek kezelését. A NearPick működése során az adatok lekérése, a foglalások kezelése vagy a képfeltöltés mind ilyen jellegű, nem blokkoló feldolgozást igénylő műveletek.

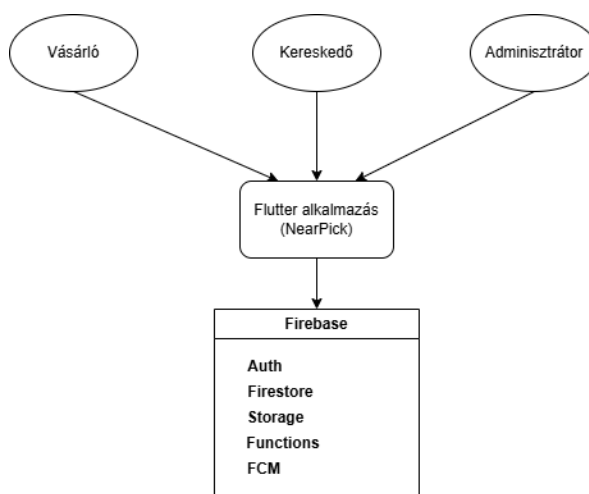
A backend megvalósításához a Firebase [2] szolgáltatásait használtam. A Firebase [2] egy felhőalapú, integrált platform, amely több különálló backend komponens helyett egységes megoldást kínál. A rendszerben a felhasználók hitelesítését a Firebase Authentication [4], az adatok tárolását a Cloud Firestore [5], a képek kezelését a Firebase Storage [6] biztosítja. A szerveroldali logika egy részének megvalósítására Cloud Functions [7] szolgál, míg az értesítések kezelését a Firebase Cloud Messaging [8] teszi lehetővé.

A fejlesztés Visual Studio Code [9] környezetben történt. A verziókezeléshez Git [13] és GitHub [14] lett használva, az automatizált ellenőrzésekhez pedig GitHub Actions [15].

4.2. A rendszer magas szintű felépítése

A felhasználók a Flutter [1] alkalmazáson keresztül lépnek kapcsolatba a rendszerrel. A kliensalkalmazás a Firebase [2] szolgáltatásaival kommunikál, amelyek a hitelesítést, adattárolást, fájlkezelést, szerveroldali műveleteket és értesítéseket biztosítják.

A rendszer három fő felhasználói szerepkört kezel: vásárló, kereskedő és adminisztrátor. Mindhárom szerepkör ugyanazt az alkalmazást használja, de a bejelentkezés után eltérő felületet és eltérő funkciókat ér el. A szerepköralapú működés miatt a rendszernek már a belépés után el kell döntenie, hogy a felhasználót melyik kezdőképernyőre irányítsa.



4.2.1 ábra: A NearPick magas szintű rendszerarchitektúrája

A Firebase Authentication [4] a felhasználó azonosítását végzi. A Cloud Firestore [5] tárolja a rendszer strukturált adatait, például a felhasználói profilokat, termékeket, foglalásokat, értékeléseket és adminisztrátori üzeneteket. A Firebase Storage [6] a termékek tárolására szolgál. A Cloud Functions [7] olyan műveleteket kezel, amelyeknél fontos a szerveroldali ellenőrzés és az üzleti szabályok betartása. A Firebase Cloud Messaging [8] az értesítési funkciókat támogatja.

Ez a felépítés serverless jellegű, vagyis a rendszer működéséhez nincs szükség külön, saját üzemeltetésű backend szerverre. Ez csökkenti az üzemeltetési terhet, miközben a

Firestore [2] szolgáltatásai elegendő alapot adnak egy több szerepkört kezelő, valós adatokkal működő mobilalkalmazás elkészítéséhez.

4.3. A Kliensalkalmazás felépítése

A NearPick kliensoldali alkalmazása szerepkörök és funkcionális területek szerint tagolt. Az alkalmazásban külön részt alkotnak az azonosításhoz, a vásárlói funkciókhoz, a kereskedői felülethez és az adminisztrátori nézetekhez tartozó képernyők. Ez a szervezés segíti az átláthatóságot, mert a különböző szerepkörök működése egymástól elkülönítve kezelhető.

A kliensoldali kódban a képernyők mellett külön szolgáltatásosztályok is találhatók. Ezek feladata, hogy kapcsolatot teremtsenek a Firestore [2] szolgáltatásokkal, és kezeljék az alkalmazás főbb műveleteit. Ilyen például a termékek lekérése és mentése, a foglalások kezelése, az adminisztrátori műveletek végrehajtása vagy az értesítésekhez szükséges tokenek mentése.

A modellek a Firestore-ban [5] tárolt adatok kliensoldali reprezentációját adják. Ilyen modell például a termék, a foglalás, a felhasználói profil vagy az értékelés. A modellek használata azért fontos, mert így az alkalmazásban nem közvetlenül, rendezetlen formában jelennek meg az adatbázisból érkező mezők, hanem strukturált objektumokként kezelhetők.

A projekt felépítése leegyszerűsítve a következő logika mentén írható le:

```
lib/  
├─ features/  
│   ├── auth/  
│   ├── consumer/  
│   ├── merchant/  
│   └─ admin/  
├─ services/  
├─ models/  
├─ recommendation/  
├─ core/  
└─ widgets/
```

Az auth rész az azonosításhoz kapcsolódó képernyőket és folyamatokat tartalmazza. A consumer mappa a vásárlói funkciókhoz, a merchant a kereskedői működéshez, az

admin pedig az adminisztrátori felületekhez kapcsolódik. A services réteg felel a Firebase-kommunikáció [2] és az üzleti műveletek jelentős részéért. A recommendation rész az ajánlatok rangsorolásához kapcsolódó logikát tartalmazza.

4.4. Hitelesítés és szerepköralapú működés

A felhasználók azonosítását a Firebase Authentication [4] végzi. Az alkalmazás email és jelszó alapú regisztrációt és bejelentkezést használ. A sikeres hitelesítés után a rendszer lekéri a felhasználóhoz tartozó profiladatokat, amelyek alapján eldönthető, hogy a felhasználó vásárlói, kereskedői vagy adminisztrátori felületre kerüljön.

Az adminisztrátori jogosultság külön kezelést igényel. A rendszerben az adminisztrátori jogosultság Firebase custom claim [16] alapján ellenőrizhető, így az admin funkciók védettebbek, mintha kizárólag egy adatbázisban tárolt szerepmező alapján működne.

4.5. Cloud Firestore és adattárolás

A Cloud Firestore [5] dokumentumalapú NoSQL adatbázisként működik. A Firestore [5] alkalmas olyan adatok kezelésére, amelyek dokumentumokként és kollekciókként jól leírhatók. A NearPick esetében ilyenek a felhasználók, termékek, foglalások, értékelések, adminisztrátori üzenetek és preferenciaadatok.

A termékek esetében az adatbázisban tárolni kell a termék nevét, kategóriáját, árát, eredeti árát, mennyiségét, lejáratát, átvételi időtartamát, helyadatát és a kereskedőhöz tartozó azonosítókat. A foglalásoknál fontos adat a vásárló, a kereskedő, a termék, a foglalt mennyiség, a foglalás státusza, valamint az átvételhez szükséges kód vagy token. Az értékelések a teljesített foglalásokhoz és a kereskedőhöz kapcsolódnak.

A Firestore-ban [5] tárolt adatok pontosabb bemutatása a következő fejezetben, az adatmodell ismertetésénél történik.

4.6. Firebase Storage és termékképek

A termékképek tárolására Firebase Storage [6] szolgál. A termékekhez tartozó képek nagyobb méretű fájlok, ezért nem lenne célszerű őket közvetlenül a Firestore [5] dokumentumokban tárolni.

A Storage [6] használatánál szintén fontos a jogosultságkezelés. Biztosítani kell, hogy csak megfelelő jogosultsággal rendelkező kereskedő tölthessen fel termékképet, és ne módosíthasson más kereskedőhöz tartozó fájlokat.

4.7. Cloud Functions és szerveroldali üzleti logika

Bár a Flutter [1] kliens több műveletet közvetlenül Firebase [2] SDK-kon keresztül végez, vannak olyan folyamatok, amelyeket biztonságosabb és megbízhatóbb szerveroldalon kezelni. Ilyenek például a foglalási műveletek, a készletmennyiség módosítása, a pickup kód vagy token kezelése, az értékelésekhez kapcsolódó frissítések, valamint az adminisztrátori műveletek egy része.

Például a foglalás létrehozása. Foglaláskor nem elegendő egy új rekordot létrehozni az adatbázisban. Ellenőrizni kell, hogy a termék aktív-e, van-e elegendő elérhető mennyiség, a felhasználó jogosult-e foglalni, majd a készletet is megfelelően csökkenteni kell. Ha ezek a lépések kizárólag kliensoldalon történének, akkor nagyobb lenne az esélye hibás működésnek.

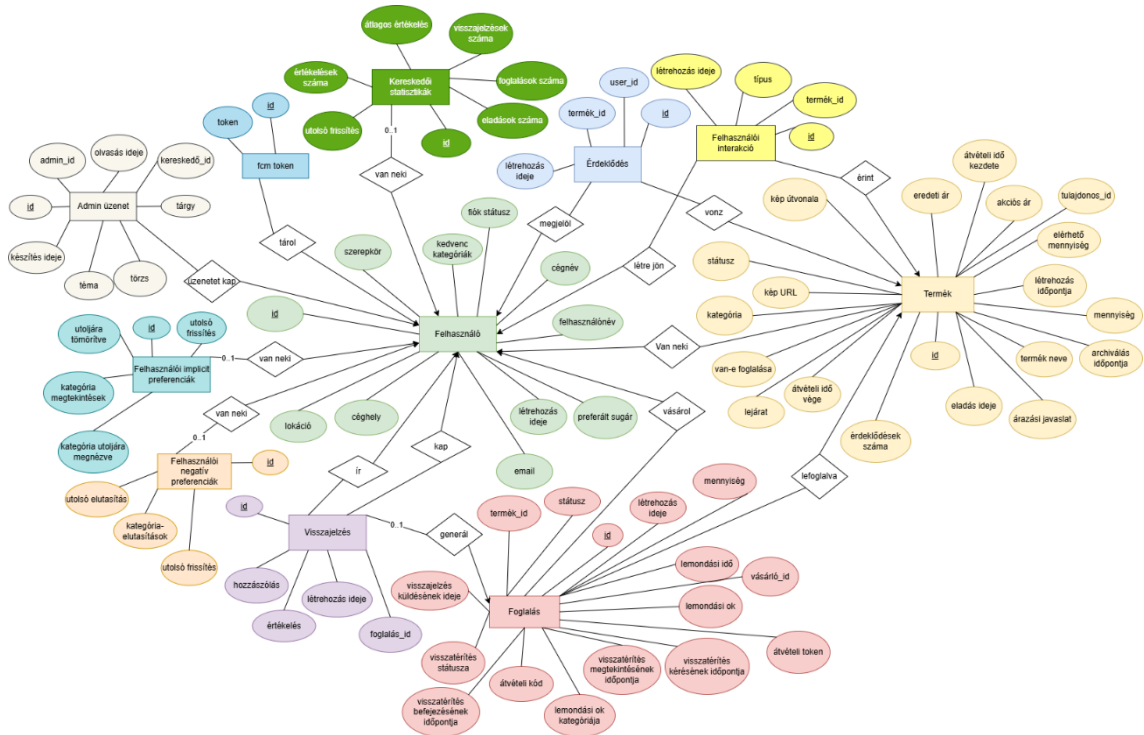
4.8. Firebase Cloud Messaging és értesítések

A Firebase Cloud Messaging [8] az értesítési funkciók megvalósítását segíti. Az alkalmazás esetében az értesítések azért fontosak, mert időérzékeny ajánlatokkal dolgozik. Egy közeli, kedvezményes termék csak rövid ideig lehet elérhető, ezért hasznos, ha a felhasználó értesítést kaphat a számára releváns új ajánlatokról.

Az értesítések működéséhez az alkalmazás elmenti a felhasználó eszközéhez tartozó FCM tokenet. Ez alapján a rendszer később értesítést küldhet az adott felhasználónak. Az értesítések küldése nem feltétlenül minden felhasználóra azonos módon történik. Figyelembe vehetők például a kedvenc kategóriák vagy más belső preferenciaadatok, így a felhasználó nagyobb eséllyel olyan ajánlatról kap értesítést, amely valóban érdekli.

Fontos, hogy az értesítések használata a felhasználói engedélyektől és az adott platform beállításaitól is függ. Ezért a NearPick nem garantálhatja, hogy minden esetben minden felhasználó értesítést kap, viszont technikailag támogatja a push értesítések küldését és kezelését.

5. Adatmodell és jogosultságkezelés



5.1 ábra: NearPick E-K diagramja

5.1. Az adatmodell felépítésének alapelvei

Az adatmodell tervezésénél az volt a cél, hogy a NearPick fő üzleti folyamatait egyszerűen leképezhető legyenek az adatbázisban is. A rendszerben meg kellett jeleníteni a felhasználókat, a kereskedők által létrehozott termékeket, a vásárlók foglaltságait, a teljesítés utáni értékeléseket, valamint az adminisztrátori beavatkozásokhoz szükséges adatokat is.

A Firestore [5] dokumentumalapú szemlélete miatt a rendszer nem klasszikus relációs adatbázisra épül, hanem kollekciók és dokumentumok formájában tárolja az adatokat. Ez lehetővé teszi, hogy az alkalmazás gyorsan lekérhető, jól strukturált dokumentumokkal dolgozzon.

Az adatmodell másik fontos alapelve a szerepköralapú elkülönítés. A vásárlóhoz, a kereskedőhöz és az adminisztrátorhoz tartozó műveletek különböző adatokkal dolgoznak, ezért az adatstruktúrának azt is támogatnia kell, hogy a felhasználók csak a számukra releváns dokumentumokhoz férjenek hozzá.

5.2. A rendszer fő entitásai

A legfontosabb entitások a felhasználók, a termékek, a foglalások és az értékelések. Ezekhez kapcsolódnak további, kiegészítő jellegű adatok, például a push értesítési tokenek, az adminisztrátori üzenetek, a kereskedői statisztikák vagy az ajánlórendszert támogató interakciós adatok.

5.2.1. Felhasználók

A felhasználók adatai a `users/{uid}` dokumentumokban jelennek meg. Egy felhasználóhoz tartozik az email-cím, a megjelenített név, a szerepkör, a fiók állapota, valamint a szerepkörtől függő további mezők. Vásárlói oldalon ilyenek lehetnek a kedvenc kategóriák, a helybeállítások vagy a preferált keresési sugár. Kereskedői oldalon a cégnév és a céghely is a profil részét képezi.

5.2.2. Termékek

A kereskedők által feltöltött ajánlatok a `products/{productId}` dokumentumokban tárolódnak. A termékhez tartozik a kereskedő azonosítója, a termék neve, kategóriája, ára, eredeti ára, elérhető mennyisége, lejárat ideje, átvételi időszaka, helyadata, valamint a termékképhez kapcsolódó elérési információk.

A termékekhez állapotinformációk is tartoznak, mivel egy ajánlat nemcsak aktív lehet, hanem elrejtett és archivált is. Archiváláskor nem tényleges fizikai törlés történik, csak a felhasználóknak nem lesz többé elérhető.

5.2.3. Foglalások

A foglalások a `reservations/{reservationId}` dokumentumokban jelennek meg. Ez az egyik legfontosabb entitás, mert a vásárló és a kereskedő közötti tényleges üzleti kapcsolatot rögzíti. Egy foglalási rekord tartalmazza a kapcsolódó termék, a vásárló és a kereskedő azonosítóját (ez jön a termékből), a foglalt mennyiséget, a foglalás státuszát, az átvételhez szükséges kódot vagy token, valamint az időbélyegeket.

A foglalásokhoz kapcsolódhatnak lemondási és visszatérítési adatok is. A rendszer rögzíti a lemondás okát, a lemondó felet, valamint a refund állapotát is.

5.2.4. Értékelések

Az értékelések a reviews/{reservationId} dokumentumokban tárolódnak. Az értékelés kapcsolódik a foglaláshoz, a kereskedőhöz, a vásárlóhoz és a termékhez is. Itt fontos hogy csak akkor lehetséges értékelést adni, ha egy foglalás már jóvá lett hagyva.

5.2.5. Kiegészítő entitások

A fő entitások mellett több kiegészítő kollekció is használatban van. Ilyen például a felhasználókhoz tartozó fcmTokens, amely az értesítésekhez szükséges tokeneket tárolja, vagy az adminMessages, amely lehetővé teszi, hogy az adminisztrátor közvetlen üzenetet küldjön egy kereskedőnek.

A kereskedői statisztikák külön merchantStats dokumentumokban vannak, így a dashboard lekérdezése egyszerűbbé válik. Az ajánlórendszert pedig olyan adatok támogatják, mint a userInteractions, userImplicitPrefs és userNegativePrefs, amelyek a felhasználói érdeklődést, megtekintési mintákat vagy elutasításokat követik.

5.3. Jogosultságkezelési modell

Természetesen meg kellett oldani, hogy a különböző felhasználók ne tudják elérni egymás funkcióit. Ehhez nem volt elég csak a frontenden elrejteni egyes gombokat, másikat meg megjeleníteni, backend oldalon is ellenőrizni kellett a jogosultságokat.

A jogosultságkezelés három fő részből áll. Az első a Firebase Authentication [4], amely biztosítja, hogy a rendszer hitelesített felhasználóval dolgozzon. A második a Firestore [5] és Storage [6] szabályrendszer, amely meghatározza, hogy az egyes dokumentumok és fájlok milyen feltételek mellett olvashatók vagy írhatóak. A harmadik a Cloud Functions [7] oldali ellenőrzés, amely a kritikus üzleti folyamatoknál további védelmet biztosít.

5.4. Firestore és Storage szabályok

A Firestore [5] szabályok célja, hogy az egyes kollekciókhoz való hozzáférés megfeleljen a rendszer üzleti logikájának. Például a felhasználó olvashatja a saját profilját, módosíthatja a saját preferenciáit, de nem írhat át más felhasználóhoz tartozó

adatot. Az adminisztrátori hozzáférésnél a szabályoknak még szigorúbbnak kell lenniük. Az admin jogosultság nem pusztán a role mező alapján dől el, hanem Firebase Auth custom claim [16] is szükséges.

A Firebase Storage [6] szabályai hasonló elv szerint működnek. A termékek tárolásánál biztosítani kell, hogy csak az adott kereskedő tölthessen fel és kezelhessen a saját termékeihez kapcsolódó fájlokat.

Művelet / adat	Vásárló	Kereskedő	Admin
Saját profil olvasása/módosítása	igen	igen	igen
Más felhasználó profiljának kezelése	nem	nem	igen
Aktív termékek olvasása	igen	igen	igen
Saját termék létrehozása/módosítása	nem	igen	korlátozottan
Más kereskedő termékének módosítása	nem	nem	igen
Saját foglalás olvasása	igen	nem	igen
Kereskedőhöz tartozó foglalások olvasása	nem	igen	igen
Admin üzenet küldése	nem	nem	igen

5.4.1 ábra: Szerepköralapú hozzáférési mátrix

6. Megvalósítás fontosabb részletei

Ebben a fejezetben a megvalósítás néhány olyan részét emelem ki, amelyek a NearPick működése szempontjából a legfontosabbak. Ezek közé tartozik az ajánlórendszer, a foglalási és készletkezelési folyamat, az átvételi kódok és QR-kódok használata, valamint az adminisztrátori műveletek technikai megoldása.

6.1. Az ajánlórendszer működése

Az alkalmazásban fontos, hogy a vásárló ne egy teljesen rendezetlen ajánlatlistát kapjon, hanem olyan termékeket lásson előrébb, amelyek nagyobb valószínűséggel relevánsak számára. Ennek érdekében kliensoldali ajánlórendszert használ, amely a termékekhez pontszámot rendel, majd ezek alapján rendezi az ajánlatokat. Nem gépi tanulási modellen alapul, hanem szabályalapú pontozáson.

A pontszámítás során figyelembe veszi a felhasználó kedvenc kategóriáit, a termék lejárat idejét, a létrehozás frissességét, a termékhez kapcsolódó érdeklődési jeleket, a felhasználó korábbi viselkedését, valamint a földrajzi távolságot. Figyeli, hogy a felhasználó mely kategóriákba tartozó ajánlatokat nyitja meg gyakrabban, és ezeket időben csökkenő súllyal számítja. A negatív visszajelzések szintén részei a logikának. Ha a felhasználó egy adott kategóriát többször elutasít vagy figyelmen kívül hagy, a rendszer ezt büntetőjelként értelmezheti, és az adott kategóriába tartozó termékek pontszámát csökkentheti. Figyelembe veszi a felhasználó helyzetét és az általa megadott preferált keresési sugarat is. Azok a termékek, amelyek ezen a sugáron belül találhatók, kedvezőbb elbírálást kapnak, míg a távolabbi ajánlatok pontszáma csökkenhet.

Ezeknek az adatoknak egy része megtekinthető az alkalmazásban is, a termék részleteinél a kis *i* ikonra kattintva. Így a vásárló is betekintést nyerhet abba, hogy miért is azt a sorrendet látja ami megjelenik.

Az összpontszám a különböző részpontszámok lineáris kombinációjaként áll elő:

$$score = clamp(w_{fav} * s_{fav} + w_{exp} * s_{exp} + w_{rec} * s_{rec} + w_{int} * s_{int} + w_{imp} * s_{imp} + w_{dist} * s_{dist} + w_{rad} * s_{rad} - p_{dismiss} - p_{outside})$$

ahol s_{fav} a kedvenc kategóriához tartozó pontszám, s_{exp} a lejárat közelség, s_{rec}

a frissesség, s_{int} az érdeklődési szám, s_{imp} az implicit viselkedési jel, s_{dist} a távolság alapú pontszám, s_{rad} pedig a preferált sugaron belüliség mértéke. A $p_{dismiss}$ és $p_{outside}$ tagok büntető komponensek.

Az implicit érdeklődés időben csökkenő súllyal szerepel a modellben. Az effektív megtekintésszám exponenciális lecsengéssel számítható:

$$effectiveCount = implicitCount * e^{-\lambda * ageDays}$$

ahol

$$\lambda = \frac{\ln 2}{halfLifeDays}$$

A NearPick jelenlegi implementációjában a felezési idő 7 nap. Ennek kódbeli megvalósítása:

```
const wFav = 0.30;
const wExp = 0.23;
const wRec = 0.12;
const wInt = 0.05;
const wImplicit = 0.10;
const wDist = 0.10;
const wRadius = 0.10;
const halfLifeDays = 7.0;
```

6.1.1 ábra: Súlyok megadása

```
final lambda = math.log(2) / halfLifeDays;
final decayFactor = math.exp(-lambda * ageDays);
final effectiveCount = implicitCount * decayFactor;

final favScore = favoriteScore(category, favoriteCategories);
final expScore = expiryScore(expiresAt, now: referenceNow);
final recScore = recencyScore(createdAt, now: referenceNow);
final intScore = interestScore(interestCount);
final implicitScore = _clamp01(effectiveCount / 10.0);
```

6.1.2 ábra: Köztes számítások

```

final score = _clamp01(
  (wFav * favScore) +
  (wExp * expScore) +
  (wRec * recScore) +
  (wInt * intScore) +
  (wImplicit * implicitScore) +
  (wDist * distanceScore) +
  (wRadius * radiusScore) -
  dismissPenalty -
  distancePenalty,
);

```

6.1.3 ábra: Végző érték kiszámítása

6.2. A foglalási folyamat és készletkezelés

Érthetően a NearPick egyik legkritikusabb művelete a foglalás létrehozása, mivel a rendszernek biztosítania kell, hogy egy termékből ne legyen több darab lefoglalható, mint amennyi ténylegesen rendelkezésre áll. Ez azért fontos, mert ugyanarra a kedvezményes ajánlatra rövid idő alatt több vásárló is reagálhat. Emiatt a foglalási folyamat nem pusztán kliensoldali adatkezelésként valósul meg, hanem szerveroldali üzleti logikára és tranzakciós feldolgozásra épül.

A kliensoldali alkalmazás a foglalás létrehozását egy Firebase [2] callable függvény meghívásával kezdeményezi. A kliens először ellenőrzi az alapvető feltételeket, például azt, hogy a felhasználó hitelesített-e, valamint hogy a megadott mennyiség érvényes-e. Ezt követően a tényleges foglalási kérés a backend felé kerül továbbításra.

```

Future<String> reserveProduct({
  required String productId,
  int quantity = 1,
}) async {
  final user = _auth.currentUser;
  if (user == null) {
    throw const AppException(
      code: 'unauthenticated',
      message: 'Bejelentkezés szükséges.',
    );
  }
  if (quantity <= 0) {
    throw const AppException(
      code: 'invalid-argument',
      message: 'Érvénytelen foglalási mennyiség.',
    );
  }

  try {
    final callable = _functions.httpsCallable('reserveProduct');
    final response = await callable.call(
      withClientContextId({'productId': productId, 'quantity': quantity}),
    );
  }
}

```

6.2.1 ábra: Foglalás kezdeményezése

A serveroldali `reserveProduct` művelet Firestore [5] tranzakcióban fut. A tranzakció először lekéri a kiválasztott termék dokumentumát, majd ellenőrzi, hogy a termék foglalható-e, van-e elegendő elérhető mennyiség, és a kérés üzleti szempontból érvényes-e. Sikeres ellenőrzés után a rendszer csökkenti a készletet, szükség esetén `sold_out` állapotba teszi a terméket, majd létrehozza a foglalási rekordot.

```
await db.runTransaction(async (tx) => {
  const productRef = db.collection("products").doc(trimmedProductId);
  const productSnap = await tx.get(productRef);
  const product = productSnap.data();
  const {ownerId, quantityAvailable, requestedQuantity} =
    assertReservableProduct(product, buyerId, quantity);

  const newQty = quantityAvailable - requestedQuantity;
  const expiresAt = admin.firestore.Timestamp.fromDate(
    new Date(Date.now() + 30 * 60 * 1000),
  );
  const pickupCode = generatePickupCode(6);
  let merchantName =
    typeof product.merchantName === "string" ? product.merchantName.trim() : "";
  if (!merchantName) {
    const merchantSnap = await tx.get(db.collection("users").doc(ownerId));
    const merchantProfile = merchantSnap.data() ?? {};
    merchantName = normalizeOptionalText(
      merchantProfile.companyName ||
      merchantProfile.displayName ||
      merchantProfile.email,
      {maxLength: 120},
    );
  }
});
```

6.2.2 ábra: Tranzakció első része

```
const productUpdates = {
  hasReservations: true,
  quantity: newQty,
  quantityAvailable: newQty,
};
if (newQty === 0) {
  productUpdates.status = "sold_out";
  productUpdates.soldOutAt = admin.firestore.FieldValue.serverTimestamp();
}

tx.update(productRef, productUpdates);
tx.set(reservationRef, {
  buyerId,
  cancelReasonCode: null,
  cancelReasonNote: "",
  cancelledBy: null,
  createdAt: admin.firestore.FieldValue.serverTimestamp(),
  expiresAt,
  merchantId: ownerId,
  pickupCode,
  pickupToken: buildPickupToken(reservationRef.id, pickupCode),
  productId: trimmedProductId,
  productSnapshot,
  qty: requestedQuantity,
  refundCompletedAt: null,
  refundRequestedAt: null,
  refundReviewedAt: null,
  refundReviewedBy: null,
  refundStatus: "not_requested",
  reviewSubmittedAt: null,
  status: "reserved",
});
```

6.2.3 ábra: Tranzakció második része

A tranzakció használata azért lényeges, mert így a készletcsökkentés és a foglalási rekord létrehozása atomi műveletként kezelhető. Ennek köszönhetően a rendszer elkerüli a túlfoglalás lehetőségét, vagyis azt az esetet, amikor több vásárló ugyanazt a készletet próbálná egy időben lefoglalni. Ha a kért mennyiség már nem érhető el, a függvény hibával tér vissza, és a foglalás nem jön létre.

A foglalási folyamat nem ér véget a létrehozással. A rendszer kezeli a lemondást is, amely során a foglalás állapota frissül, és a korábban lefoglalt mennyiség visszakerülhet az elérhető készletbe.

```
const currentAvailable =
  Number.isInteger(product.quantityAvailable) ?
    product.quantityAvailable :
    Number.isInteger(product.quantity) ? product.quantity : 0;
const incrementBy = Number.isInteger(reservation.qty) ? reservation.qty : 1;
const newAvailable = currentAvailable + incrementBy;
const productExpiresAt =
  typeof product.expiresAt?.toDate === "function" ?
    product.expiresAt.toDate() :
    null;
const nextStatus =
  productExpiresAt && productExpiresAt.getTime() <= Date.now() ?
    "expired" :
    newAvailable > 0 ? "active" : (product.status ?? "active");

tx.update(productRef, {
  quantity: newAvailable,
  quantityAvailable: newAvailable,
  status: nextStatus,
});
tx.update(reservationRef, {
  cancelReasonCode: reasonCode,
  cancelReasonNote: reasonNote,
  cancelledAt: admin.firestore.FieldValue.serverTimestamp(),
  cancelledBy: "buyer",
  refundCompletedAt: null,
  refundRequestedAt: refundRequested ?
    admin.firestore.FieldValue.serverTimestamp() :
    null,
  refundReviewedAt: null,
  refundReviewedBy: null,
  refundStatus: refundRequested ? "pending" : "not_requested",
  status: "cancelled",
});
```

6.2.4 ábra: Termék lemondása

A termék új állapota a visszaállított mennyiség és a lejáratí idő figyelembevételével kerül meghatározásra. Ha a termék még nem járt le és ismét van belőle elérhető mennyiség, akkor a státusza visszaállhat aktív állapotra.

A foglalási életciklus utolsó pontja a teljesítés. Az átvétel is szerveroldalon kezeli van kezelve. A teljesítés előtt a backend ellenőrzi, hogy a foglalás valóban az adott kereskedőhöz tartozik-e, a státusza megfelelő-e, és a megadott pickup adat érvényes-e.

```

await db.runTransaction(async (tx) => {
  const reservationRef = db.collection("reservations").doc(reservationId.trim());
  const reservationSnap = await tx.get(reservationRef);
  const reservation = reservationSnap.data();

  assertCompletableReservation(
    { ...reservation, id: reservationRef.id },
    merchantId,
    pickupInput,
  );

  tx.update(reservationRef, {
    completedAt: admin.firestore.FieldValue.serverTimestamp(),
    status: "completed",
  });
});

```

6.2.5 ábra: Foglalás befejezése

A NearPick foglalási és készletkezelési megoldása tehát három fő elvre épül: a kliensoldal csak kezdeményező szerepet tölt be, a kritikus műveletek szerveroldali callable függvényeken keresztül futnak, a készletváltozások pedig tranzakcióban történnek.

6.3. QR-kódos és átvételi kódos teljesítés

A foglalási folyamat fontos része az átvétel megbízható azonosítása. Két formája van neki: QR-kódos és manuális kódos teljesítés. A rendszer nemcsak egyszerű szöveges azonosítót használ, hanem strukturált pickup tokenet is létrehoz, ami a foglalás azonosítóját és a pickup kódot egyaránt tartalmazza.

A backend oldalon a pickup token előállítás a következő formában történik:

```

function generatePickupCode(length = 6) {
  const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
  let result = "";
  for (let i = 0; i < length; i += 1) {
    result += chars[Math.floor(Math.random() * chars.length)];
  }
  return result;
}

function buildPickupToken(reservationId, pickupCode) {
  return `NEARPICK:${reservationId}:${pickupCode}`;
}

```

6.3.1 ábra: Kód generálása

Ahogy látható, a token formátuma a következőképpen írható le:

token = NEARPICK:reservationId:pickupCode

Ez azért lett így kialakítva, mert a QR-kód nem egy tetszőleges karakterláncot tartalmaz, hanem strukturált módon hordozza a foglalás azonosításához szükséges adatokat. A manuális átvételi kód ezzel párhuzamosan egyszerűbb tartalékmegoldást ad olyan esetekre, amikor a QR-kód beolvasása valamilyen okból nem működik.

A kliensoldalon a rendszer képes megkülönböztetni a sima pickup kódot és a teljes QR tokenből származó bemenetet. A feldolgozás logikája a következő:

```
ParsedPickupToken parsePickupToken(String rawInput) {
    final value = rawInput.trim();
    if (value.isEmpty()) {
        throw const FormatException('Üres átvételi kód.');
```

```
    }

    if (!value.startsWith('NEARPICK:')) {
        return ParsedPickupToken(reservationId: null, pickupCode: value);
    }

    final parts = value.split(':');
    if (parts.length != 3 || parts[1].isEmpty() || parts[2].isEmpty()) {
        throw const FormatException('Érvénytelen QR token.');
```

```
    }

    return ParsedPickupToken(reservationId: parts[1], pickupCode: parts[2]);
}
```

6.3.2 ábra: Kód értelmezése

Ebből látszik hogy kétféle bemenetet tud értelmezni. Ha nem NEARPICK előtaggal kezdődik, akkor azt hagyományos átvételi kódként értelmezi, egyébként külön kinyeri a foglalás azonosítóját és a pickup kódot.

A tényleges teljesítés azonban nem történhet kizárólag kliensoldali ellenőrzéssel. A kereskedő által megadott vagy beolvasott adatot a rendszer szerveroldalon is validálja. A backend ellenőrzi, hogy a foglalás létezik-e, az adott kereskedőhöz tartozik-e, még nem járt-e le, és a megadott pickup adat valóban megfelel-e a foglaláshoz tartozó kódnak.

```
function assertCompletableReservation(reservation, callerUid, pickupInput) {
    if (!reservation) {
        throw new Error("not-found");
    }

    if (!callerUid || reservation.merchantId !== callerUid) {
        throw new Error("permission-denied");
    }

    if (reservation.status !== "reserved") {
        throw new Error("invalid-status");
    }

    const expiresAt = asDate(reservation.expiresAt);
    if (expiresAt && expiresAt.getTime() <= Date.now()) {
        throw new Error("expired-reservation");
    }

    const parsed = parsePickupInput(pickupInput);
    if (parsed.reservationId && parsed.reservationId !== reservation.id) {
        throw new Error("invalid-pickup-code");
    }
    if (reservation.pickupCode !== parsed.pickupCode) {
        throw new Error("invalid-pickup-code");
    }
}
```

6.3.3 ábra: Kód ellenőrzése

A rendszer nem engedi teljesíteni azokat a foglalásokat, amelyek már nem reserved állapotúak, vagy lejártak, valamint a QR tokenben szereplő foglalásazonosítónak és pickup kódnak is egyeznie kell az adatbázisban tárolt értékekkel.

A szerveroldali ellenőrzés után tranzakcióban módosítódik a foglalás állapota, ahogy az a 6.2.4-es ábrán is látható.

6.4. Adminisztrátori műveletek megvalósítása

Az admin műveletek alapja egy központi ellenőrző függvény, amely minden érzékeny callable művelet előtt lefut. Ez először azt vizsgálja, hogy a kérés hitelesített és aktív fióktól érkezik-e, majd ellenőrzi, hogy a felhasználó rendelkezik-e adminisztrátori jogosultsággal.

```
async function assertAdminRequest(request, contextId) {
  await assertActiveCaller(request, contextId);

  if (request.auth?.token?.admin !== true) {
    throw new HttpsError("permission-denied", "Admin jogosultság szukseges.", {
      contextId,
    });
  }
}
```

6.4.1 ábra: Admin ellenőrzés

Látható, hogy Firebase Auth custom claim [16] ellenőrzés is használva van, ez csökkenti annak a kockázatát, hogy egy felhasználó kliensoldali manipulációval admin felülethez jusson.

Az egyik legfontosabb admin művelet a felhasználói fiókok állapotának módosítása. A lehetséges állapotok az aktív, felfüggesztett és blokkolt. A státuszváltás során a backend nemcsak a Firestore [5] profiladatot módosítja, hanem szükség esetén a Firebase Authentication [4] szintjén is letiltja a felhasználót. Emellett a rendszer auditmezőket is rögzít arról, hogy ki és mikor végezte a módosítást.


```

exports.setUserAccountStatus = onCall(async (request) => {
  const contextId = createContextId(request);
  logInfo("admin.user_status.started", {contextId});

  await assertAdminRequest(request, contextId);

  const userId = request.data?.userId;
  const accountStatus = normalizeOptionalText(request.data?.accountStatus, {maxLength: 24});
  if (typeof userId !== "string" || userId.trim().length === 0) {
    throw new HttpsError("invalid-argument", "Ervenytelen userId.", {contextId});
  }
  if (!VALID_ACCOUNT_STATUSES.has(accountStatus)) {
    throw new HttpsError("invalid-argument", "Ervenytelen accountStatus.", {contextId});
  }
  if (userId.trim() === request.auth.uid && accountStatus !== "active") {
    throw new HttpsError("failed-precondition", "A saját admin fiok nem tilthato vagy fuggesztheto fel innen.", {contextId});
  }

  await admin.auth().updateUser(userId.trim(), {
    disabled: accountStatus === "blocked",
  });
  await admin.firestore().collection("users").doc(userId.trim()).set({accountStatus,
    statusUpdatedAt: admin.firestore.FieldValue.serverTimestamp(), statusUpdatedBy: request.auth.uid}, {merge: true});
});

```

6.4.2 ábra: Státusz módosítása

A termékmoderáció szintén fontos funkció. Nem fizikai törlés van alkalmazva elsődleges megoldásként, hanem állapotalapú moderáció. Egy adminisztrátor elrejthet terméket, majd később vissza is állíthatja annak korábbi állapotát, ehhez a rendszer eltárolja a termék eredeti státuszát is.

```

exports.hideProductForAdmin = onCall(async (request) => {
  const contextId = createContextId(request);
  logInfo("admin.product_hide.started", {contextId});

  await assertAdminRequest(request, contextId);

  const productId = request.data?.productId;
  if (typeof productId !== "string" || productId.trim().length === 0) {
    throw new HttpsError("invalid-argument", "Ervenytelen productId.", {contextId});
  }

  const productRef = admin.firestore().collection("products").doc(productId.trim());
  const snap = await productRef.get();
  if (!snap.exists) {
    throw new HttpsError("not-found", "A termék nem található.", {contextId});
  }

  const product = snap.data() ?? {};
  const currentStatus =
    typeof product.status === "string" && product.status.trim().length > 0 ? product.status.trim() : "active";
  const statusBeforeHidden = currentStatus === "hidden" ?
    (typeof product.statusBeforeHidden === "string" && product.statusBeforeHidden.trim().length > 0 ?
      product.statusBeforeHidden.trim() : "active") : currentStatus;

  await productRef.set({status: "hidden", statusBeforeHidden}, {merge: true});
});

```

6.4.3 ábra: Termékmoderáció

Ahogy fent írtam is, a statusBeforeHidden mezőben van eltárolva az előző állapot, így visszaállításakor nem kell feltételezni, hogy a termék korábban feltétlenül aktív volt.

A visszaállítás ehhez illeszkedve a mentett állapotot használja fel:

```
const product = snap.data() ?? {};  
const restoreStatus =  
  typeof product.statusBeforeHidden === "string" &&  
    product.statusBeforeHidden.trim().length > 0 ?  
    product.statusBeforeHidden.trim() :  
    "active";  
  
await productRef.set({  
  status: restoreStatus,  
  statusBeforeHidden: admin.firestore.FieldValue.delete(),  
}, {merge: true});
```

ábra 6.4.4: Termék állapotának visszaállítása

Az adminisztrátori kommunikáció szintén külön backend műveletként valósul meg. Az admin közvetlen üzenetet küldhet egy kereskedőnek, amely a címzett adminMessages alkollekciójába kerül. A backend ellenőrzi, hogy a célfelhasználó valóban kereskedő-e, majd elmenti az üzenetet, és szükség esetén push értesítést is küld.

```
const merchantProfile = merchantSnap.data() ?? {};  
if (merchantProfile.role !== "merchant") {  
  throw new HttpsError("failed-precondition", "Admin  üzenet csak kereskedőnek küldhető.", {contextId});  
}  
  
const adminProfile = await getUserProfile(request.auth.uid);  
const createdByLabel = normalizeOptionalText(adminProfile.displayName || adminProfile.email || "Admin",  
  {maxLength: 80}) || "Admin";  
  
const messageRef = merchantRef.collection("adminMessages").doc();  
await messageRef.set({body, createdAt: admin.firestore.FieldValue.serverTimestamp(), createdBy: request.auth.uid,  
  createdByLabel, readAt: null, recipientUserId: merchantId.trim(), subject, topic});  
  
const tokenSnap = await merchantRef.collection("fcmTokens").get();  
const tokens = tokenSnap.docs  
  .map((doc) => doc.data().token)  
  .filter((token) => typeof token === "string" && token.length > 0);  
  
if (tokens.length > 0) {  
  const pushBody = body.length > 140 ? `${body.slice(0, 137)}...` : body;  
  await admin.messaging().sendEachForMulticast({  
    notification: {title: `Admin  üzenet: ${subject}`, body: pushBody},  
    data: {messageId: messageRef.id, topic, type: "admin_message"},  
    tokens  
  });  
}
```

6.4.5 ábra: Adminisztrátori üzenetküldés

Ez biztosítja hogy az üzenet csak kereskedőnek küldhető. Az üzenet létrehozása szerveroldali műveletként történik, így a kliens nem tud tetszőleges admin üzenetet létrehozni. Az értesítési tokenek felhasználásával az üzenet nemcsak az adatbázisban jelenik meg, hanem közvetlen push értesítéssel is jelezhető.

7. Tesztelés és minőségbiztosítás

A megvalósított funkciók mellett szinte párhuzamos készültek a tesztek, hogy a legfontosabb üzleti folyamatok ellenőrizhetők legyenek. A tesztelés több szinten történt: kliensoldali egység- és widget tesztek, workflow jellegű integrációs tesztek, valódi UI-folyamatokat ellenőrző integration_test futásokat, valamint backend és jogosultsági tesztek is.

A projektben 96 Flutter [1] tesztdefiníció és 71 Functions [7] vagy rules teszt található, vagyis összesen 167 automatizált teszteset-definíció védi a rendszer különböző részeit.

7.1. Kliensoldali tesztelés

A Flutter [1] oldalon az egységtesztek a fontosabb üzleti logikákat és segédkomponenseket ellenőrzik. Külön tesztek készültek például az ajánlórendszerhez, a földrajzi távolságszámításhoz, a termék- és foglalásmodellekhez, a location preference kezeléshez, a pickup token feldolgozáshoz, a dinamikus árazási javaslatokhoz és a dashboard metrikákhoz.

A widget tesztek a felhasználói felületet vizsgálják. Külön tesztek találhatók például a bejelentkezési és regisztrációs képernyőkhöz, a szerepköralapú root routinghoz, valamint a kereskedői termékfeltöltő képernyőhöz. Ezek azt ellenőrzik, hogy a felület megfelelő állapotban jelenik-e meg, a validációs hibák megjelennek-e, és a főbb navigációs döntések helyesen történnek-e meg.

Az integrációs tesztek nem egyetlen osztályt vagy widgetet, hanem összefüggő alkalmazásfolyamatokat vizsgálnak. Külön workflow tesztek futnak az azonosítási folyamatra, a piactér működésére, a termékkezelésre és a foglalási életciklusra. Ezek in-memory fake környezetre támaszkodnak, így gyorsabban és determinisztikusabban futtathatók, mint a teljes UI-alapú tesztek.

7.2. Backend és jogosultsági tesztek

Mivel sok fontos művelet Cloud Functions [7] callable végpontokon keresztül fut, ezért a backend oldali ellenőrzés is fontos volt. Például a foglalás létrehozása, a teljesítés,

a lemondás, a refund státusz kezelése, az értékelés beküldése, valamint az adminisztrátori státusz- és moderációs műveletek.

A `security_helpers.test.js` a kritikus döntési pontokat ellenőrzi, például hogy milyen feltételek mellett foglalható, teljesíthető vagy review-zható egy foglalás. A `notification_segments.test.js` az értesítési szegmentálás működését teszteli. A `firestore_rules_contract.test.js` és a `firestore_rules_policy.test.js` a hozzáférési szabályok viselkedését vizsgálják, például azt, hogy ki hozhat létre vagy módosíthat terméket, és ki olvashat foglalási adatokat. Az `admin_callable_policy.test.js` az adminisztrátori callable műveletekhez kapcsolódó engedélyezési, validációs és happy path eseteket ellenőrzi. Itt jelenik meg például a nem admin hívók tiltása, a saját admin fiók blokkolásának megakadályozása, a termékmoderációs műveletek ellenőrzése, valamint az admin üzenetküldéshez kapcsolódó logika.

7.3. Automatizált minőségkapuk

Több automatizált quality gate is szerepel, amelyek a fejlesztés során és a CI-folyamatban is futtathatók. A Flutter [1] oldalon ilyen a formázási ellenőrzés, a statikus analízis és a teljes tesztsomag futtatása. Emellett dependency audit is elérhető, amely a használt csomagok ismert sebezhetőségeit vizsgálja.

A backend oldalon külön JavaScript quality gate-ek működnek. Ilyen a lintelés, a Functions [7] tesztek futtatása és a függőségek biztonsági ellenőrzése. Repo szinten secret scan is szerepel, amely segít kiszűrni az esetlegesen véletlenül bekerülő érzékeny adatok vagy kulcsok mintáit. A lokális quality gate-ek scriptből is futtathatók, így a fejlesztés közben is egyetlen belépési pontból ellenőrizhető a projekt állapota.

8. Mesterséges intelligencia használata a fejlesztés során

A szakdolgozat elkészítése során tudatosan kellett használni mesterséges intelligencián alapuló eszközöket, ezzel tapasztalva, hogy milyen mértékben tudják ezek az eszközök támogatni egy szoftverfejlesztési folyamat különböző szakaszait. Az MI használat nem csak kódrészletek generálását jelentette, hanem kiterjedt a technológiai döntések előkészítésére, az architektúra átgondolására, a dokumentáció készítésére, hibakeresésre, valamint a szakdolgozat szerkezetének megtervezésére is.

A projekt alapötlete tölem származott, az első MI használat a technológiai döntésnél történt. A ChatGPT-nek [17] leírtam az alkalmazás tervezett működését, a főbb funkciókat, valamint azt is, hogy körülbelül mennyi idő áll rendelkezésre. Kikötöttem azt is, hogy amennyiben mobilalkalmazás készül, nem szeretném Android Studio-ban készíteni. Ez alapján a Flutter [1] keretrendszert javasolta a kliensalkalmazáshoz, a Firebase [2] szolgáltatásokat pedig backendként, adatbázisként és hitelesítési megoldásként.

A Fluttert [1] korábban nem ismertem, ezért utánanéztem a működésének, lehetőségeinek, és végigcsináltam egy rövid tutorialt is. Ezek alapján jó választásnak tűnt, így elfogadtam. A Firebase [2] használata kevésbé volt új, mivel korábbi projektekben már dolgoztam vele, ezért ez is jó ötletnek tűnt. Az alkalmazás nevének kiválasztásánál szintén használtam MI-t: a projekt rövid leírása alapján kértem névjavaslatokat, és ezek közül alakult ki a NearPick elnevezés.

A fejlesztés korai szakaszában elsősorban ChatGPT-t [17] és Claude Sonnet 4.x-et [18] használtam. Ezeket egyszerű chatfelületen keresztül alkalmaztam: leírtam az adott funkciót vagy problémát, majd a kapott kódrészleteket beillesztettem a projekt megfelelő részeibe. Ez a módszer kezdetben működött, de hamar kiderült, hogy nagyobb projekt esetén nem ideális. Nem látták át az egész kódbázist, ezért a kontextushány megnehezítette a pontos fejlesztést. Ekkor kezdtem el a Codex [19] használatát Visual Studio Code [9] környezetben, lokálisan a teljes repóval együtt.

A Codex [19] használata nagy változást hozott a fejlesztési folyamatban. Mivel hozzáfért a projekt fájlszerkezetéhez és a meglévő kódhoz, sokkal célzottabban tudott módosításokat javasolni és végrehajtani. Elsőként kisebb feladatokkal próbáltam ki, majd fokozatosan egyre nagyobb fejlesztési egységeket bíztam rá. A Codexet [19] használtam

kódgenerálásra, refaktorálásra, hibakeresésre, dokumentációs fájlok létrehozására, tesztek előkészítésére és a projekt szerkezetének átnézésére is.

A fejlesztés során az MI által adott kimeneteket nem tekinthettem végleges megoldásként, mert többször előfordult hogy a generált kód hibás volt, nem fordult le, vagy csak futás közben jött elő a hiba. Gyakori probléma volt még hogy a Codex [19] bizonyos Dart [3] vagy Flutter [1] parancsokat, illetve tesztfuttatásokat nem tudott megfelelően lefuttatni a saját környezetében, ezeket nekem kellett külön terminálból elvégezni, majd az eredmények alapján folytatni a javítást.

A validáció több szinten történt. Egyrészt majdnem minden módosítás után manuálisan kipróbáltam az alkalmazás érintett funkcióját. Másrészt használtam a Flutter [1] és Dart [3] által biztosított ellenőrző parancsokat, és a már elkészült teszteket.

Az MI-t nemcsak a programkódnál, hanem a repó dokumentációjának elkészítésénél is használtam. Egy megadott sablon alapján kértem hogy listázza ki, milyen dokumentumokra van szükség. Ezeket átnéztem, majd a megfelelő sorrendben, kisebb egységekben kértem az elkészítését és bővítését. Ebben szerintem kifejezetten jó az MI, a repó alapján jó README fájlokat, és különböző projekt specifikus dokumentumokat tud készíteni.

A szakdolgozat szövegének megírásánál az MI-t elsősorban ötletelésre, szerkezettervezésre használtam. Kértem javaslatokat arra, hogy az alkalmazás mely részeit érdemes külön fejezetekben bemutatni, hogyan lehetne kiemelni a fontosabb technikai döntéseket. A szöveg kidolgozását azonban nem bíztam az MI-re. Amikor egyes fejezetek kidolgozását kértem tőle, azt tapasztaltam hogy nagyon sokat ismétli önmagát, valamint olyan általánosságokat ír, ami egyáltalán nem fontos a dolgozat szempontjából. Az ábrák és diagramok elkészítésénél is igénybe vettem. A Codex [19], mivel látta a projekt fájljait és felépítését, ezeket elég jól el tudta készíteni. Egy-két helyen pontatlan volt, ezeket külön javítottam.

A folyamat tanulsága az volt, hogy az MI jelentősen fel tudja gyorsítani a fejlesztést, de nem váltja ki a fejlesztői gondolkodást. Úgy tűnhet hogy egyszerűbbé válik a munka, mert kevesebb kódot kell írni, viszont épp ezért más típusú figyelmet igényel. Pontosan kell megfogalmazni a feladatokat, követni kell a változtatásokat, ellenőrizni kell a generált kódot, és érteni kell az MI döntéseit. Ha ez elmarad, akkor könnyen előfordulhat, hogy a projekt bár működik, de a fejlesztő nehezen látja át a saját rendszerét, ami későbbi problémákhoz és nehézségekhez vezethet.

9. Továbbfejlesztési lehetőségek

A NearPick jelenlegi állapotában egy működő prototípusként mutatja be, hogyan lehet a helyi kereskedők időérzékeny, kedvezményes termékeit összekapcsolni a közelben tartózkodó vásárlókkal. Ez több olyan irányban is továbbfejleszthető, ami egy éles környezetben működő szolgáltatás esetén fontos lenne.

Az egyik legfontosabb ilyen funkció az online fizetés bevezetése. A jelenlegi működésben a vásárló lefoglalja a terméket az alkalmazásban, majd az átvételkor, helyben fizet a kereskedőnek. Egy éles rendszerben azonban hasznos, ha a vásárló már az alkalmazáson belül ki tudná fizetni a terméket. Ezt a már meglévő logikához jól illeszthető lenne, valahogy így nézne ki: A fizetési folyamat során a Flutter [1] alkalmazás elküldené a rendelési vagy foglalási szándékot a Firebase [2] backend felé, például Cloud Functions [7] használatával. A szerveroldali logika hozná létre a fizetési tranzakciót a választott fizetési szolgáltatónál, majd a kliensalkalmazás megnyitná a szolgáltató fizetési felületét vagy SDK-ját. A rendeltetést csak akkor jelölné fizetett állapotúnak a Firestore [5] adatbázisban, ha a fizetési szolgáltató webhookon keresztül visszaigazolta a sikeres fizetést.

Ehhez kapcsolódó továbbfejlesztési irány a visszatérítési folyamat tényleges megvalósítása. Jelenleg a visszatérítési állapotok nyilvántartási jelleggel már megjelennek, de online fizetés hiányában ezek nem kapcsolódnak valódi pénzügyi tranzakcióhoz. Ha az alkalmazás később fizetési szolgáltatóval egészülne ki, akkor a lemondások, elutasított átvételek vagy kereskedői problémák esetén a rendszernek kezelnie kellene a részleges vagy teljes visszatérítéseket is.

Egy másik nagyobb fejlesztési irány a harmadik féllel történő integráció, például futárszolgálat bevonása. A NearPick alapvetően helyi átvételre épül, de bizonyos esetekben hasznos lehetne házhozszállítási lehetőséget is biztosítani. A futárszolgálati integráció már nagyobb módosítást igényelne a működésben. A rendelés létrehozása után a backendnek ellenőriznie kellene, hogy a rendelés fizetett-e, a termék átvethető-e, valamint rendelkezésre áll-e a szükséges készlet. Ezután a futárszolgáltató API-ján keresztül létrehozna egy szállítási igényt. A futárszolgáltató visszaadna egy csomagazonosítót, követési számot vagy címkenyomtatási URL-t, amelyet a NearPick eltárolna a Firestore [5] adatbázisban. A kereskedő ezután előkészítené a csomagot, a

vásárló pedig az alkalmazásban követhetné a szállítás állapotát, például „feladva”, „szállítás alatt” vagy „kézbesítve” állapotokkal.

További fejlesztési lehetőség a felhasználói felület és a felhasználói élmény finomítása, a jelenlegi verzióban a hangsúly elsősorban a működő üzleti folyamatokon volt.

Összességében a NearPick fejlesztése során sikerült egy könnyen használható, működőképes és bővíthető prototípust létrehozni. Az alkalmazás a legfontosabb folyamatokat már jelenlegi állapotában is lefedi. Bár az éles, nagyobb léptékű bevezetéshez további fejlesztésekre, szélesebb körű tesztelésre és publikálási előkészületekre lenne szükség, a rendszer jelenlegi formájában is alkalmas lehetne egy kisebb városi vagy helyi közösségi próbaüzem alapjaként.

Irodalomjegyzék

- [1] Flutter documentation: <https://docs.flutter.dev/>
- [2] Firebase for Flutter: <https://firebase.google.com/docs/flutter>
- [3] Dart documentation: <https://dart.dev/docs>
- [4] Firebase Authentication: <https://firebase.google.com/docs/auth>
- [5] Cloud Firestore: <https://firebase.google.com/docs/firestore>
- [6] Cloud Storage for Firebase: <https://firebase.google.com/docs/storage>
- [7] Cloud Functions for Firebase: <https://firebase.google.com/docs/functions>
- [8] Firebase Cloud Messaging: <https://firebase.google.com/docs/cloud-messaging>
- [9] Visual Studio Code: <https://code.visualstudio.com/>
- [10] Too Good To Go: <https://www.toogoodtogo.com/en-us>
- [11] Munch: <https://munch.eco/hu/fooldal/>
- [12] FoodHero: <https://foodhero.com/>
- [13] Git: <https://git-scm.com/>
- [14] GitHub: <https://github.com/>
- [15] GitHub Actions documentation: <https://docs.github.com/en/actions>
- [16] Firebase custom claims: <https://firebase.google.com/docs/auth/admin/custom-claims>
- [17] ChatGPT: <https://openai.com/chatgpt/overview/>
- [18] Claude Sonnet: <https://www.anthropic.com/claude/sonnet>
- [19] Codex: <https://openai.com/index/codex-now-generally-available/>

Nyilatkozat

Alulírott Kálmán Ádám programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel.

Tudomásul veszem, hogy szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

2026.05.03

Aláírás

Elektronikus mellékletek

1. Az alkalmazás GitHub elérhetősége: <https://github.com/SZTE-SZF/1-sprint-Admgt>
2. Az alkalmazás hosztolt linkje: <https://nearpick-c0fea.web.app/>